Stacklyst

Manual técnico e funcionamento da plataforma

Laboratório de Engenharia de Software
Revisão de 3 de novembro de 2026
Base de código 6a2bfce e documentação local revisada

Pedro Antonio Silvestre Tescaro
Guilherme Alves de Campos
Paulo Henrique Leal dos Santos

Esta edição atualiza os modelos e a especificação do sistema. Os diagramas de atividades do negócio aprovados permanecem no documento 02, preservado integralmente. Os modelos revisados são editáveis no Astah UML. Cadastro e acesso usam o ator Usuário e declaram as pré-condições de sessão em cada caso.

As afirmações descrevem a fonte de implementação. Credenciais, infraestrutura externa, dispositivos e migrations dependem das verificações de ambiente indicadas. Esta revisão documental não representa execução de todos os fluxos em produção.

Roteiro de leitura

Roteiro de leitura 2
Trilhas por dentro: do currículo ao progresso persistido 6
1. O que cada camada representa 6
2. Entrada na interface 7
3. Disponibilidade do grafo 7
4. Acesso a uma lição 7
5. Contrato de resposta 8
6. Avaliação por tipo 8
7. Prévia e submissão 9
8. Primeira conclusão e transação 9
9. Exercícios persistidos 9
10. Ajuda e domínio 9
11. XP, nível e painéis 10
12. Credenciais 10
13. Como acrescentar uma lição 10
14. Diagnóstico prático 10
Mundinho por dentro: mundo, economia e combate 11
Estrutura 11
Economia: três registros diferentes 11
Baú e check-in 12
Inventário e equipamento 12
Criaturas e níveis 12
Golpes contra criaturas 12
Arena: entrada e convite 13
Presença e movimento 13
Início da partida e equipamentos 13
Turno de PvP 14
Dano, cura e encerramento no PvP 14
Apresentação e aparência 14
Arte e mobile 14
Pontos de manutenção 15
Sequência técnica do Mundinho 1 16
Sequência técnica do Mundinho 2 17

Sequência técnica do Mundinho 3 18
Comunidade, identidade e recursos pessoais 19
Página inicial e feed 19
Consultas e personalização 19
Respostas e threads 19
Reações, votos e enquetes 19
Mídia e música 20
Perfil e identidade visual 20
Conquistas, títulos, badges e certificados 21
Relações sociais e descoberta 21
Ranking 21
Notificações 21
Bate-papo 21
Itens salvos 22
Configurações e administração 22
Quiz diário 22
Denúncias 22
Upload de imagens 22
Comunidade editorial 23
Avaliação de código, duelos e revisão humana 23
Contratos independentes 23
Execução de código 23
Harness e testes 24
Convite e compromisso competitivo 24
Executar e submeter 24
Ordem da decisão automática 24
Limites da análise 25
Persistência e recompensas 25
Candidatura a avaliador 25
Reputação e autorização para revisar 25
Verificações recomendadas 25
Vagas, recrutamento, eventos e hackathons 26
Empresa e autorização 26
Listagem e detalhe da vaga 26

Criação de vaga 26
Candidatura 27
Movimentação do funil 27
Estados não equivalem a ordem obrigatória universal 27
Eventos e hackathons 27
Inscrição e capacidade 27
Desafios do evento 28
Dependência de ambiente 28
Dados adequados para demonstração 28
Contratos, segurança, falhas e operação 28
Caminho de uma requisição 28
Identidade e banco 29
Contratos públicos e internos 29
Erros HTTP 29
Concorrência e idempotência 30
Rede e recuperação 30
Rate limit, logs e auditoria 30
Banco, migrations e seeds 30
Testes e significado das evidências 31
Mobile 31
Manutenção da documentação 31
Código comentado e rastreável 32
apiHandler 32
requireAuth 33
deriveKnowledgeStatus 33
calculatePathProgress 34
requireLessonAccess 34
assessLessonStep 35
publicLesson 36
saveLessonAssessment 36
evaluateExerciseCode 37
recordExerciseSubmission 38
recordAssistanceEvent 39
calculateExerciseXp 40

deriveCompletedNodeStatus 40
prepareCode 40
executeCode 41
awardXPInTransaction 42
calculateLevel 43
reconcileCredentialsInTransaction 43
getEarnedCredentials 44
getPublicCredential 45
create 45
getFeed 46
visiblePostWhere 47
toggleReaction 47
toggleFollow 48
getUserNotifications 49
markAllAsRead 50
saveProfileCustomization 50
dayKey 51
resolveTurn 52
equipmentStats 52
credit 53
adventure 53
validArenaPosition 54
chooseArenaAttack 54
advanceArenaTurn 55
arena 56
chooseBestDuelSubmission 57
decideDuelResolution 57
awardDuelWinnerXP 58
checkEligibility 59
submitDuelEvaluation 60
requireCompanyAccess 61
applyForJob 61
updateApplicationStage 62
participate 63

replaceSet 64
planCommunity 65
publishCommunity 65
Referência de UML e critérios de modelagem 66
Casos de uso e sua especificação 66
Herança dos atores 67
Inclusão, extensão e pontos de extensão 67
Classes e persistência 67
Sequências e ligação com o código 68
Uso no seminário 68

O sumário é atualizado na exportação pelo Word. As fontes editáveis e a matriz de rastreabilidade acompanham o pacote.

Trilhas por dentro: do currículo ao progresso persistido

1. O que cada camada representa

As trilhas combinam duas organizações de conteúdo. O currículo de lições organiza sequências de atividades com IDs e pré-requisitos. O grafo de conhecimentos organiza nós e caminhos compartilhados. O usuário pode enxergar ambos por componentes de mapa/jornada; isso não torna seus registros intercambiáveis.

Elemento	Responsabilidade	Fonte principal
Curso	Agrupar uma experiência por linguagem/tema	learning/catalog-types.ts, catálogo
Lição	Reunir etapas, objetivos e pré-requisitos	learning/catalog.ts, foundations/specializations
Etapa	Definir uma interação avaliada ou explicação	lessons/types.ts
Nó de conhecimento	Representar competência no grafo	KnowledgeNode
Aresta	Expressar relação obrigatória, recomendada ou relacionada	KnowledgeEdge
Caminho	Selecionar e ordenar nós	LearningPath, LearningPathNode
Exercício	Definir problema, função, testes e política de ajuda	Exercise, ExerciseTestCase
Execução	Guardar uma prévia executada	ExerciseRun
Submissão	Guardar uma avaliação enviada para conclusão	ExerciseSubmission

Elemento	Responsabilidade	Fonte principal
Progresso	Registrar situação do usuário no nó	UserNodeProgress
Tentativa de etapa	Vincular usuário à resposta de uma etapa	QuizAttempt, por compatibilidade histórica

O nome QuizAttempt pode induzir a erro: a tabela também guarda acertos de etapas do currículo. Um relatório precisa usar o ID e a origem para distinguir quiz diário, lição e recibos especiais.

2. Entrada na interface

A página trails/page.tsx [../../../src/app/trails/page.tsx] prepara a experiência. TrailsContent coordena seleção e apresentação; os componentes de mapa, caminho e jornada apresentam os dados. Preferências de curso/modo e o tamanho disponível podem mudar qual visualização aparece.

Não se deve tomar uma animação ou um nó colorido como autoridade do progresso. O componente recebe estado do domínio e traduz esse estado para apresentação. Para investigar uma tela desatualizada, confira a resposta da API e a transformação de dados antes de mudar regras pedagógicas.

3. Disponibilidade do grafo

A função deriveKnowledgeStatus [../../../src/lib/learning/progress.ts] recebe estado persistido, estados de requisitos obrigatórios e indicação de próximo recomendado.

1. Se o estado persistido é MASTERED, COMPLETED ou IN_PROGRESS, preserva-o.

2. Caso ainda haja requisito obrigatório sem conclusão/domínio, deriva NOT_STARTED.

3. Com requisitos satisfeitos, deriva RECOMMENDED para a próxima indicação.

4. Nos demais casos, deriva AVAILABLE.

Isso evita apagar um histórico por uma nova recomendação. Também mostra por que “recomendado” não significa “obrigatório”: são decisões diferentes.

calculatePathProgress conta nós em COMPLETED ou MASTERED, divide pelo total e arredonda o percentual. Caminho vazio retorna 0%, não divisão por zero. A próxima indicação é o primeiro nó ainda não concluído na ordem do caminho. O mesmo nó em percursos diferentes não deve exigir duas conclusões independentes.

4. Acesso a uma lição

requireLessonAccess [../../../src/lib/learning/lesson-progress.ts] consulta as etapas corretas das lições pré-requisito e da própria lição. Se a lição atual já está completa legitimamente, permite revisão mesmo após alteração do currículo. Caso contrário, a ausência de pré-requisitos gera LESSON_LOCKED com status 403.

Há uma distinção importante: uma etapa existir no catálogo não implica que toda conta pode submetê-la imediatamente. A rota deve resolver a lição, localizar a etapa e aplicar a regra de acesso antes da avaliação.

5. Contrato de resposta

A API de tentativa [../../../src/app/api/lessons/%5BlessonId%5D/attempt/route.ts] exige sessão e aplica limite de 30 pedidos por minuto por usuário nesse fluxo. lessonAnswerSchema limita os dados:

Campo	Uso	Limite atual
stepId	Identificar etapa da lição	1–200 caracteres
action	Distinguir run de submit	Padrão submit
selectedOption	Escolha objetiva	Inteiro 0–30, quando informado
code	Solução de código	Até 20.000 caracteres
blanks	Respostas de lacunas	Valor de até 1.000 caracteres
tokens	Blocos ordenados	Até 100 tokens de até 1.000 caracteres
order	Ordem de IDs	Até 100 itens de até 200 caracteres
pairs	Correspondência	Valor de até 1.000 caracteres
command	Atividade de terminal	Até 2.000 caracteres

O schema valida forma e limites. Depois disso, a etapa correta precisa ser encontrada na lição e avaliada segundo seu tipo. Um payload válido ainda pode conter uma resposta errada.

6. Avaliação por tipo

assessLessonStep [../../../src/lib/learning/assessment.ts] não trata tudo como impressão de texto.

Tipo	Critério efetivo
concept_explanation	Conteúdo explicativo; resultado de avaliação positivo não o torna uma atividade obrigatória equivalente às demais
multiple_choice / output_prediction	Índice escolhido igual ao índice esperado
code_completion	Cada lacuna precisa coincidir com uma alternativa esperada, após trim
drag_drop	Sequência de tokens igual à sequência esperada
ordering	Ordem de IDs igual à ordem dos itens por correctIndex
matching	Cada chave esquerda deve apontar ao correspondente direito
terminal	Comando após trim igual ao comando esperado
code_editor / debug / boss_challenge	Execução contra contrato de avaliação ou contra saída explicitamente configurada

Uma etapa de código sem testes aplicáveis ou saída esperada não deve ser considerada concluída apenas porque o programa rodou. A rota distingue indisponibilidade do executor e responde com RUNNER_UNAVAILABLE/503 antes de persistir progresso.

7. Prévia e submissão

Testes ocultos entram no contrato de submissão quando definidos. Em run, a prévia filtra os casos públicos. Console e saída são feedback, não recibo de conclusão.

8. Primeira conclusão e transação

saveLessonAssessment [../../../src/lib/learning/lesson-progress.ts] abre learningTransaction e usa o lock learning:<userId>, compartilhado com outras recompensas pedagógicas. Isso protege tanto dois envios da mesma etapa quanto créditos de atividades diferentes concorrendo sobre o total do usuário.

A função consulta tentativas da lição, identifica a etapa e calcula firstCompletion = isCorrect && !previous?.is_correct. O XP é max(0, step.xp) apenas nessa primeira conclusão. Uma tentativa anteriormente errada pode passar a correta e receber o crédito elegível; uma já correta não volta a render XP.

Quando necessário, cria o registro Quiz correspondente à etapa. A tentativa é criada ou promovida a correta. O crédito é aplicado com awardXPInTransaction, e a resposta reúne xpEarned, firstCompletion, completedStepIds, lessonCompleted e totalXp.

Etapas obrigatórias para fechamento excluem concept_explanation. A lição é considerada completa quando todas as etapas restantes estão corretas e existe pelo menos uma etapa obrigatória. Se houve uma nova conclusão que fecha a lição, a reconciliação de credenciais ocorre no fluxo transacional.

9. Exercícios persistidos

O subsistema em exercises/repository.ts [../../../src/lib/exercises/repository.ts] tem acesso, workspace, execução, submissão e eventos de ajuda próprios. Ele não deve ser confundido com a API de tentativa de lição, embora compartilhe execução e conceitos de progresso.

requireExerciseAccess confere o nó e requisitos. getExerciseForEvaluation pode buscar casos ocultos para uso interno; a versão pública não deve entregá-los. recordExerciseRun registra a prévia. recordExerciseSubmission valida o resultado, serializa a recompensa e registra primeira conclusão. recordAssistanceEvent registra ajuda conforme o contrato.

A primeira conclusão é um fato persistido (first_completion), não uma inferência de que “esta foi a primeira vez que vi o botão verde”. Progresso, XP e relatórios devem usar esse fato sem contar repetidamente cada execução de código.

10. Ajuda e domínio

Os modos são GUIDED, STANDARD, HARD e NO_ASSIST. No cálculo atual de progression.ts [../../../src/lib/exercises/progression.ts], todos multiplicam XP por 1 e adicionam 0 ao domínio. Isso impede documentar bônus maiores apenas porque o modo se chama difícil.

O domínio calculado usa proporção de exercícios concluídos, limitada a 100. A derivação de conclusão retorna IN_PROGRESS quando faltam exercícios e COMPLETED quando o conjunto é concluído. O comentário da implementação ressalta que ajuda autodeclarada não mede domínio independente.

11. XP, nível e painéis

awardXPInTransaction atualiza o total e a linguagem quando aplicável, além das movimentações por período usadas pelo ranking. Um valor negativo não deve ser enviado ao helper geral como se fosse recompensa. Fluxos de reversão têm tratamento próprio.

O currículo declara o tipo da etapa e pode personalizar XP. Os painéis devem usar evidências confirmadas, não estimar conclusão a partir da quantidade de cliques. O mesmo vale para “Meus estudos”: preferência declarada de estudo e atividade efetivamente concluída são informações diferentes.

12. Credenciais

credentials/catalog.ts [../../../src/lib/credentials/catalog.ts] define metas e requisitos. O repositório confere etapas corretas e exercícios aprovados; a credencial emitida guarda definição e versão, evidências e dados de integridade. Requisitos antigos legitimamente alcançados permanecem consultáveis por snapshot após evolução do currículo.

Consultar credencial pública não deve conceder uma nova. Gerar PDF representa o registro válido encontrado, com QR apontando à verificação. Título de perfil, XP e arte do certificado não substituem a conferência das evidências exigidas.

13. Como acrescentar uma lição

1. Defina o objetivo observável e o curso correto.
2. Escolha ID estável da lição e confira como defineLesson produz IDs de etapa.
3. Declare pré-requisitos existentes e evite ciclos.
4. Escolha tipos de interação e configure seus gabaritos/contratos internos.
5. Separe exemplos públicos de casos ocultos relevantes.
6. Confira a serialização de publicLesson para não expor soluções.
7. Teste caminho correto, incorreto, repetido, bloqueado e indisponibilidade.
8. Confira efeito em requisitos de credenciais, histórico e visualização mobile.

Não se deve renumerar etapas publicadas apenas para organizar melhor o arquivo sem analisar registros existentes. Compatibilidade de IDs é parte da manutenção pedagógica, não detalhe de apresentação.

14. Diagnóstico prático

Sintoma	Investigar primeiro
Lição aparece trancada	Pré-requisitos no catálogo e tentativas corretas que o servidor encontrou

Sintoma	Investigar primeiro
Código imprime e não passa	Contrato da função, tipo do retorno e casos ocultos
Resposta passa, mas XP é zero	run, repetição correta, XP configurado e elegibilidade
Outra pessoa acessa atividade bloqueada	Autorização e requisitos na rota/repositório, não somente a tela
Percentual do caminho parece errado	Conjunto de nós, compartilhamento e estados considerados concluídos
Certificado não aparece	Meta concreta, primeira conclusão, reconciliação, versão e validade
Atualização do currículo mudou o histórico	IDs, legacyStepIds, requisitos e snapshots

Os testes de catálogo, progresso, persistência de lições e avaliação ficam nos diretórios __tests__ das áreas correspondentes. Os recortes de implementação [codigo-comentado.md] mostram trechos exatos para acompanhar o fluxo sem depender de nomes memorizados.

Mundinho por dentro: mundo, economia e combate

Estrutura

AdventureHub [../../../src/components/mundinho/AdventureHub.tsx] coordena vistas como entrada, clareira, inventário, batalha e arena. World [../../../src/components/mundinho/World.tsx] apresenta cenário, movimento, interação e personagens. O domínio está em src/lib/mundinho; a API autenticada está em /api/mundinho.

As ações formam uniões discriminadas: cada objeto tem type e os campos compatíveis com aquela ação. O schema Zod do repositório valida limites e alternativas. Assim, comprar, equipar, responder e mover não compartilham um payload livre sem contrato.

Economia: três registros diferentes

Dado	Responsabilidade
RpgPlayer.coins	Saldo atual do jogador
RpgTransaction	Histórico de créditos/débitos com chave por usuário
sourceCoins	Marco da reconciliação de moedas originadas da aprendizagem

A função credit usa a unicidade usuário/chave para que o mesmo evento não seja creditado duas vezes. Atualizar apenas o saldo sem registrar a origem quebraria a possibilidade de verificar repetição e extrato.

O repositório reconcilia conclusões elegíveis de exercícios e tentativas do currículo. O valor-base observado nesse fluxo é cinco moedas por atividade contada; compara a origem acumulada com sourceCoins e credita somente a diferença positiva. Não concede cinco moedas por qualquer clique ou execução repetida.

Baú e check-in

O dia é calculado por dayKey no fuso America/Sao_Paulo. O baú da clareira concede 10 moedas com chave chest:forest:<dia>. Pedidos simultâneos/repetidos para o mesmo dia encontram a mesma chave. Há compatibilidade com o recibo legado do primeiro baú quando ele foi resgatado naquele dia.

Check-in tem lastCheckin, sequência e ciclo de recompensas próprios. Não é outro nome para o baú. Pular dias pode alterar a sequência calculada; reabrir a tela não cria outro dia.

Na interface, confirmação, abertura e estado resgatado são fases diferentes. Se o servidor não confirmou crédito, a animação não pode afirmar que as moedas foram recebidas. A fonte do sprite e sua exportação estão separadas do extrato financeiro do RPG.

Inventário e equipamento

Compra procura o item nos catálogos de cosméticos ou equipamentos e calcula o preço no servidor. A posse é representada no inventário; equipar exige compatibilidade de slot. Um ID enviado livremente pelo cliente não concede o item.

equipmentStats considera somente IDs presentes no inventário e no slot correto. Calcula vida máxima, proteção, sorte, golpe direto e drop. Os limites atuais incluem vida-base 100 com bônus de vitalidade limitado a 50 e proteção limitada a 8.

Cálculo	Regra atual
Golpe direto comum	min(100, 75 + sorte)
Exercício	100 - chanceDireta
Caça-bugs possuído/equipado	Golpe direto 100%
Drop	Base 50% mais sorte, em fluxo próprio

Sorte é expressa em pontos percentuais. Um bônus de 10 leva a chance direta de 75% a 85%, não a 82,5%. A chance de exercício cai para 15%. O sorteio de equipamento ainda depende do pool/raridade/nível; não é a mesma operação do sorteio de exercício.

Criaturas e níveis

O mapa declara criaturas e posições. A batalha salva a identidade do encontro e seus números, incluindo nível e vida máxima do inimigo. O nível do jogador, derivado do XP oficial, é usado para acesso e apresentação; não deve substituir o nível do encontro.

O catálogo inclui níveis 1, 2, 3, 4, 5 e 10. creatureStats calcula vida máxima 100 + max(0, nível - 3) * 20. Assim, nível 10 tem 240 PV. O limite de turnos de criatura também escala: max(12, nível * 2 + 4).

A função canChallengeCreature compara nível do jogador e da criatura. Mensagem da interface ajuda a entender o bloqueio; a proteção relevante também precisa existir na ação do servidor. O catálogo e o snapshot tornam consistente o nível mostrado no mundo e o usado no combate.

Golpes contra criaturas

Os golpes são definidos em SKILLS:
Revisão de 3 de novembro de 2026 12

ID	Nome	Dano ao alvo em acerto	Efeito no atacante
strike	Aplicar conceito	30	Nenhum bônus próprio
focus	Depurar	18	Cura até 20 PV
burst	Refatorar	45	Desgaste de 8 PV

No combate contra criatura, resolveTurn também calcula o contra-ataque do inimigo quando ele sobrevive: dano conforme acerto/erro, nível e proteção. Não se deve aplicar automaticamente essa mesma rotina de contra-ataque ao PvP, que tem alternância explícita de jogadores.

Os cenários de código avaliam movimento/traço e estruturas exigidas. Inserir o nome de uma estrutura em comentário não comprova seu uso. Instrumentação e reprodução do traço ajudam o aluno a ver o que sua solução realmente executou.

Arena: entrada e convite

O portal leva a uma sala privada para dois participantes. O anfitrião consulta amigos mútuos e convida uma conta elegível; o convidado aceita pela interface. Bloqueio e perda da relação exigida são verificados no servidor. Convite tem validade de cinco minutos.

RpgArenaRoom persiste participantes, convite, presença, posições, versão e partida. A API deriva nome, avatar completo, linguagem e nível das fontes autorizadas. O pacote de movimento contém posição/direção/estado de caminhada, não uma nova identidade ou inventário.

O pop-up reúne o adversário e as ações de convite/prontidão/revanche, deixando o mapa usar a mesma janela da clareira. Abrir o diálogo pausa a caminhada; fechar devolve o foco pela interação de diálogo do navegador.

Presença e movimento

A implementação utiliza snapshots HTTP. O próximo pedido é programado após o anterior, em vez de iniciar requisições sem limite. A sincronização periódica do cliente usa aproximadamente 600 ms entre conclusões de pedidos; a lista social tem atualização menos frequente. Aba oculta suspende o polling correspondente.

O servidor confere área caminhável e deslocamento compatível com o tempo. O cliente interpola outros avatares. Respostas com versão antiga da mesma sala são ignoradas para não fazer o mundo “voltar no tempo”. HTTP polling e interpolação visual não são uma conexão de jogo em tempo real sem latência.

Início da partida e equipamentos

Ambos precisam confirmar prontidão. Ao iniciar, o servidor consulta inventário/equipamento dos dois, calcula os bônus, salva stats e inicializa os PV conforme vida máxima. Trocar equipamento fora desse snapshot não muda retroativamente a partida. A revanche obtém um novo snapshot.

O sorteio do primeiro jogador e dos golpes é feito no servidor. A ação de ataque informa sala, partida, turno e habilidade. Pedidos de turno antigo não repetem dano.

Turno de PvP

1. Conferir se partida está ativa e o solicitante é o ator do turno.
2. Escolher strike, focus ou burst.
3. Sortear golpe direto ou exercício com os bônus capturados.
4. Se direto, aplicar o efeito e alternar turno.
5. Se exercício, guardar exerciseTurn e a habilidade escolhida.
6. Executar é prévia; submeter solução válida resolve o turno.
7. Repetir ataque após o sorteio não troca habilidade nem refaz a chance.

A compilação externa ocorre fora da transação de sala. Ao retornar, o servidor confere novamente partida, ator e turno. Isso protege contra uma solução que terminou depois de expiração, saída ou outro fechamento.

Dano, cura e encerramento no PvP

O dano do golpe é reduzido pela proteção do adversário, com mínimo 1. Depurar cura somente no acerto, respeitando o máximo; Refatorar aplica desgaste próprio. Falha/expiração não aplica os efeitos de acerto. O desgaste pode reduzir a vida do próprio atacante a zero.

Cada turno tem 120 segundos. Zero PV encerra; ao chegar ao limite de 20 turnos, a comparação de PV define vencedor ou empate. Saída e desconexão possuem resolução própria no serviço. Um jogador offline há mais que a tolerância não pode manter indefinidamente a partida.

PvP não entrega moedas ou XP. São partidas de prática e convivência; recompensas da economia, de criaturas e de duelos competitivos devem ser documentadas separadamente.

Apresentação e aparência

O campo de batalha usa battle-background.png, também usado nas criaturas. Para cada observador, seu personagem fica embaixo à esquerda olhando à direita, e o oponente acima à direita olhando para baixo. O referencial é o observador, não o anfitrião.

O avatar preserva roupa, pele, cabelo, cor personalizada, olhos, gênero e acessórios conforme o contrato. Nome sobre o personagem usa nome de exibição autorizado. A aparência do próprio perfil social não é necessariamente o mesmo objeto do personagem RPG.

Arte e mobile

O LibreSprite é usado na produção/exportação de recursos pixel art; fontes editáveis ficam em artifacts/mundinho, e arquivos de produção em public/assets/mundinho. Atlas, células, direções e transparência devem continuar compatíveis com os componentes consumidores.

No mobile, o build do Mundinho gera o documento embarcado e inclui arte. A ponte nativa encaminha operações autenticadas. Uma alteração de componente/asset exige regenerar o documento, mas isso não substitui gerar aplicativo instalável ou testar um aparelho físico.

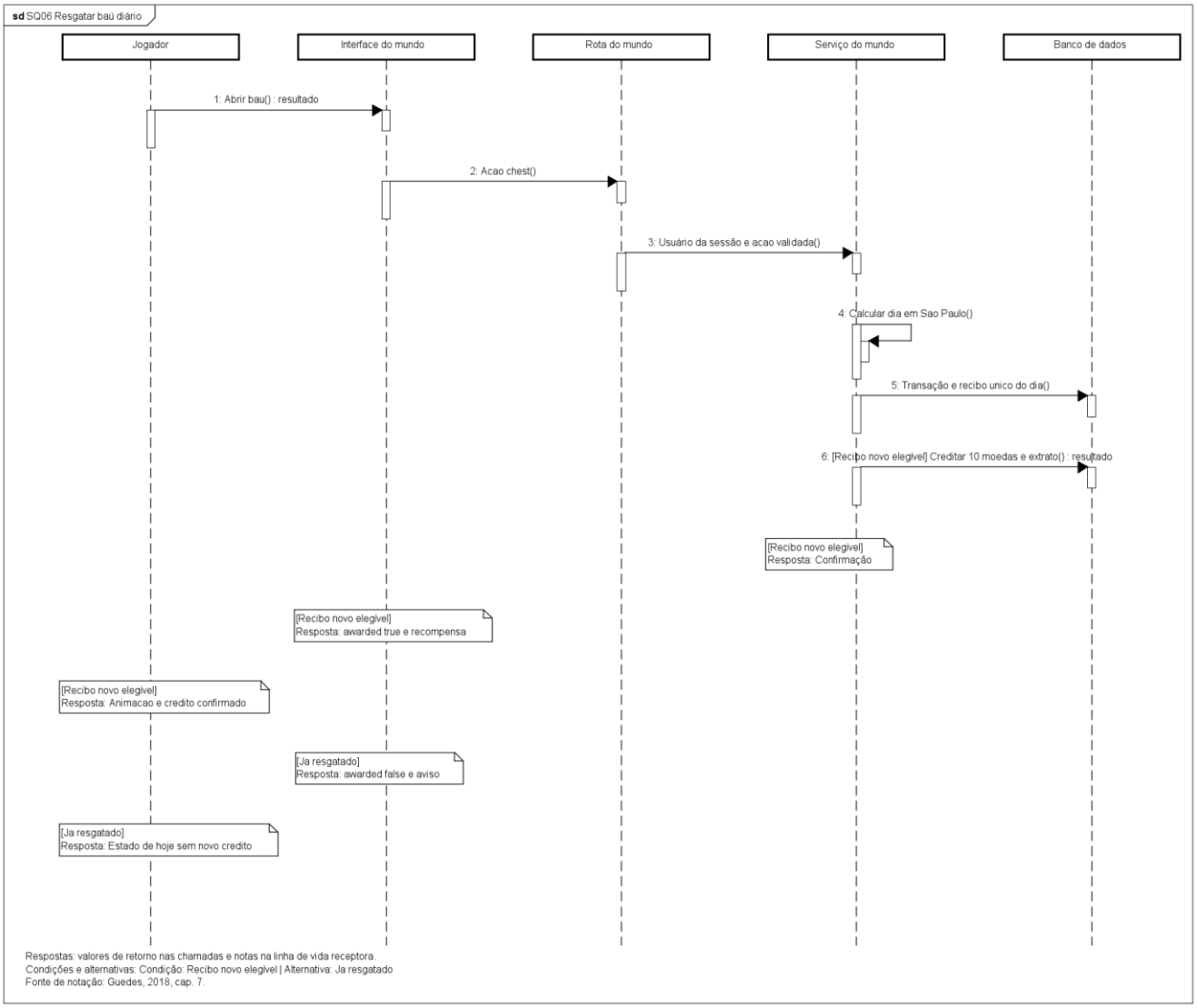
Pontos de manutenção

Ao alterar probabilidades, teste limites do sorteio e não apenas um exemplo. Ao alterar golpe, confira dano, cura, desgaste, proteção, morte própria, logs e ambos os modos. Ao alterar presença, teste pedidos antigos, aba oculta, reconexão e saída. Ao alterar economia, confira unicidade, saldo, extrato e eventos legados.

Fontes: repositório RPG [../../../src/lib/mundinho/repository.ts], equipamentos [../../../src/lib/mundinho/equipment.ts], regras [../../../src/lib/mundinho/rules.ts], arena [../../../src/lib/mundinho/arena.ts], regras da arena [../../../src/lib/mundinho/arena-rules.ts], protocolo documentado [../../quality/mundinho-multiplayer.md].

Sequência técnica do Mundinho 1

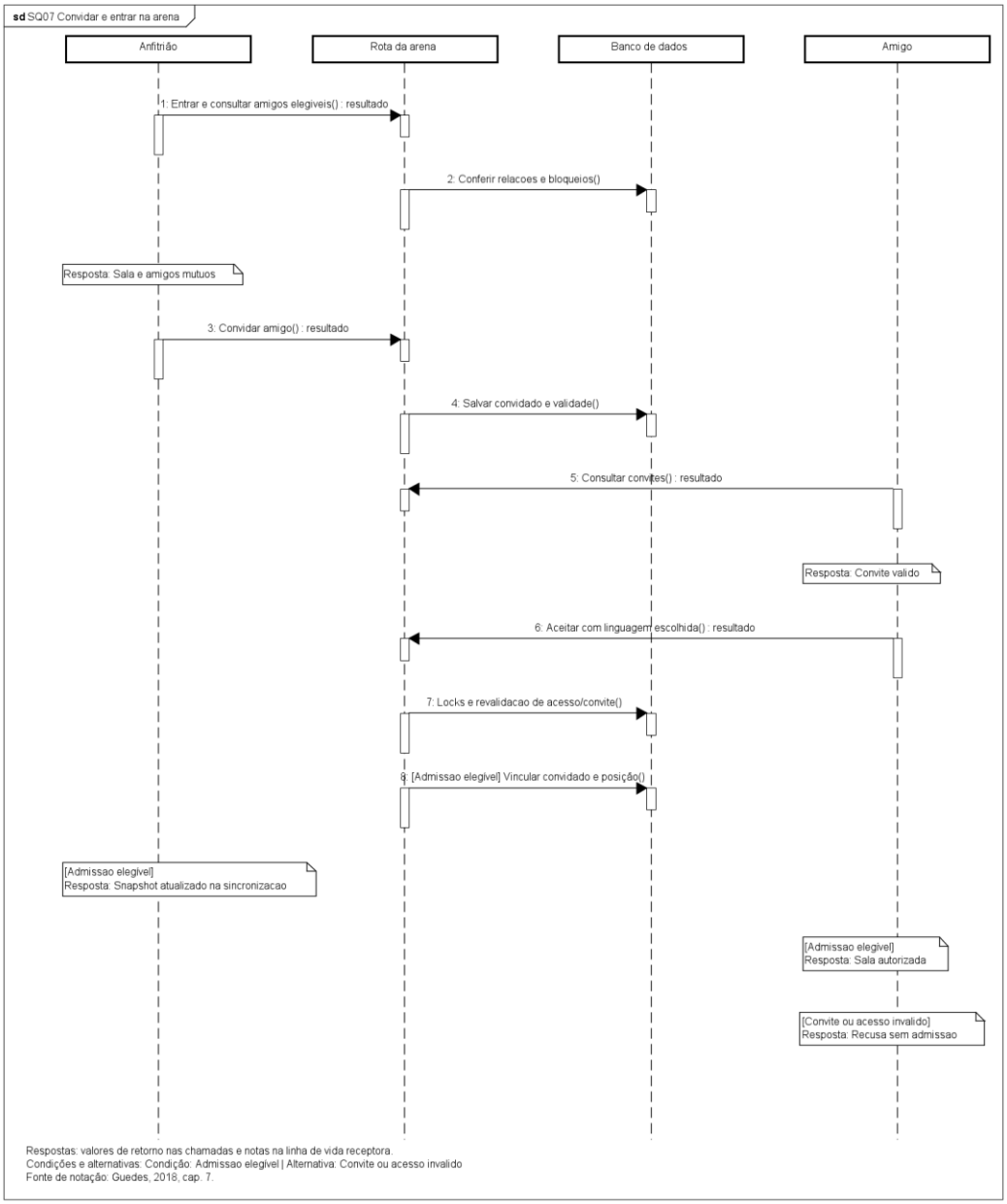
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



SQ06 Sequência técnica do Mundinho 1

Sequência técnica do Mundinho 2

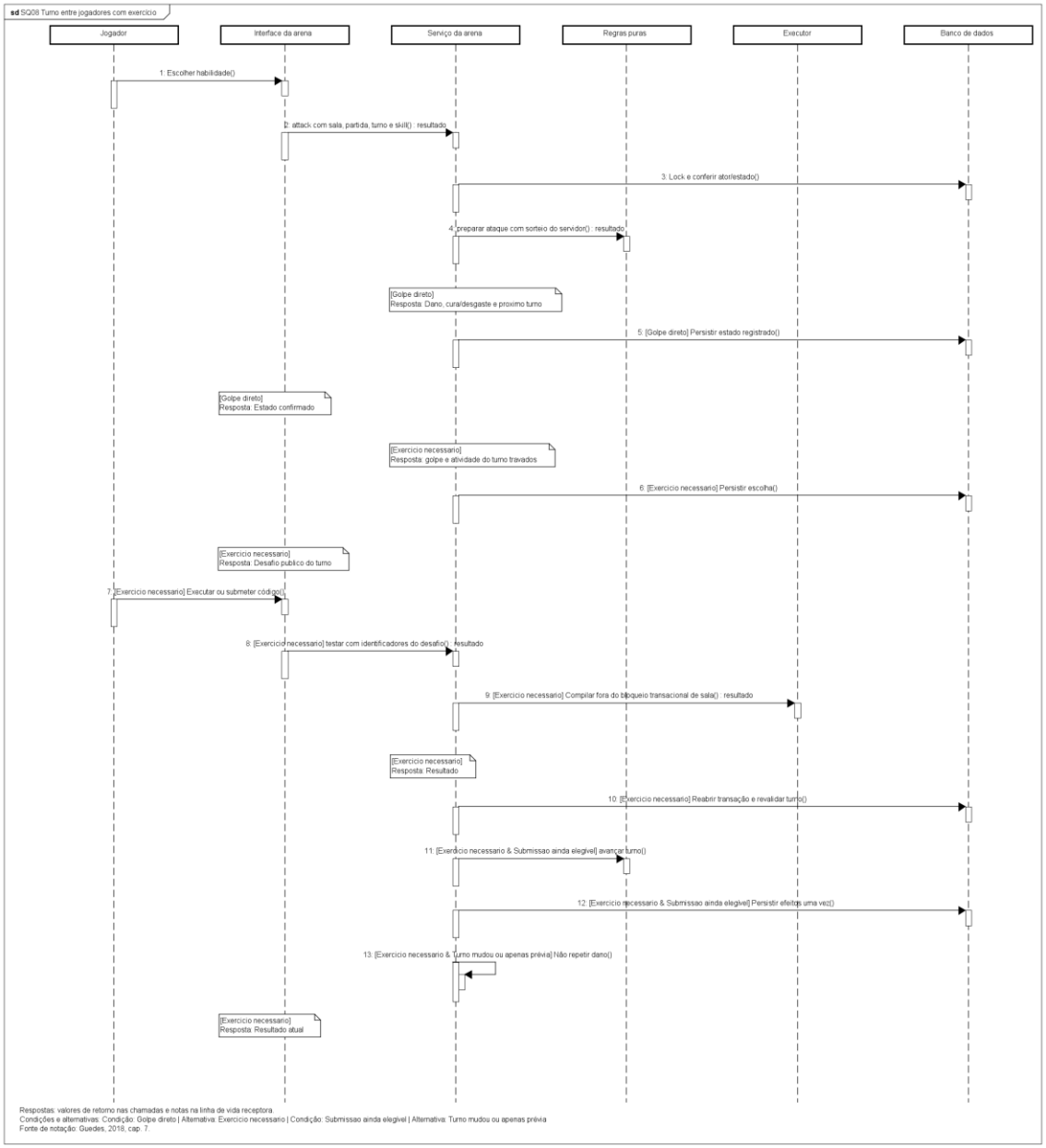
Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



SQ07 Sequência técnica do Mundinho 2

Sequência técnica do Mundinho 3

Fonte: projeto nativo Stacklyst-modelos-atualizados.asta. Amplie a imagem ou abra a vista específica para ler os detalhes.



SQ08 Sequência técnica do Mundinho 3

Comunidade, identidade e recursos pessoais

Página inicial e feed

A raiz pública e o feed autenticado são superfícies diferentes. src/app/page.tsx resolve identidade inicial para HomeClient. A área de feed reúne publicações e filtros. A escolha de uma tela pelo cliente não substitui acesso autenticado onde ele é exigido.

PostService [../../../src/services/post.service.ts] organiza criação e consultas. Na criação, a combinação autor/client_id permite reconhecer um pedido repetido. Recursos de mídia são conferidos contra registros do proprietário: uma URL arbitrária não deve equivaler a upload autorizado.

O serviço cria o post e seus efeitos de banco; a interface deve tratar espera, erro e resposta confirmada. Salvar a prévia de uma imagem no estado React não significa que o arquivo terminou de ser enviado ou está disponível para publicação.

Consultas e personalização

getFeed aceita busca, linguagem, autor, filtro de seguidos, ordenação, cursor e consultas por interações. O filtro de seguidos usa relações do usuário atual; usuário sem sessão não recebe uma lista fictícia de pessoas seguidas. visiblePostWhere aplica audiência e visibilidade do autor antes de entregar o conteúdo.

O feed geral recente pode reordenar a página consultada por afinidade de linguagens e relações seguidas. Isso não significa carregar a base inteira para decidir o ranking. Ordenação popular no conjunto de seguidos possui cursor próprio. Não misture os cursores de diferentes critérios como se representassem a mesma posição.

Para um post ausente, examine: registro existente → audiência → bloqueios → filtros → cursor → ordenação da página. Remover autorização para “fazer aparecer” é um erro de manutenção.

Respostas e threads

Answer aponta ao post, ao autor e opcionalmente a uma resposta-pai. O campo parent_id produz o encadeamento. A página consulta respostas de raiz e inclui replies em níveis determinados; a interface limita a indentação e apresenta respostas conforme sua posição.

Uma thread editorial usa post (1/N), resposta (2/N) e continuações ligadas à anterior. A ordem visual depende também de created_at. Numerar mensagens sem configurar os vínculos não produz o mesmo encadeamento.

O aceite de uma resposta e os votos têm contratos próprios. Responder não concede o direito de editar o post original. O autor da mensagem, o autor do post e o moderador são papéis diferentes na operação.

Reações, votos e enquetes

O catálogo de reações de post aceita FIRE, HEART, LAUGH, CLAP e BULB. A associação usuário/post é única. Selecionar o mesmo tipo não cria outra linha; trocar tipo atualiza a reação; remover trata a reversão conforme a recompensa daquele fluxo.

O serviço de reação serializa a transição com alterações de XP do usuário. Na reversão observada, o decréscimo é limitado para não tornar XP negativo, e a movimentação de período e linguagem é mantida coerente.

Voto em post, reação e voto em opção de enquete são dados diferentes. Uma porcentagem da enquete deve derivar das opções e votos autorizados, não da quantidade de reações do post. Reações em mensagens também possuem tabela e endpoints próprios.

Mídia e música

O editor reúne texto, código e anexos conforme contrato. O domínio valida propriedade dos anexos e compatibilidade de tipo. As apresentações de fotos/vídeos/áudio/GIF e música usam componentes específicos; um post com música não exige ser uma submissão de código.

Integrações de música dependem de disponibilidade e configuração do provedor. Consultar uma busca ou resolver um link não representa licença de reprodução irrestrita. O manual descreve o caminho implementado, sem afirmar disponibilidade permanente do serviço externo.

Perfil e identidade visual

O perfil combina conta, nome de exibição, bio, informações opcionais e preferências. username é identificador de URL; nome de exibição é apresentação e pode ter espaços, símbolos e estilo. Não utilize nome de exibição como chave única de autorização.

profile-style.ts [../../../src/lib/profile-style.ts] normaliza identidade compartilhada por avatar, nome e título. profile-customization.ts [../../../src/lib/profile-customization.ts] declara schemas e catálogo; o serviço de personalização [../../../src/services/profile-customization.service.ts] confere dados, disponibilidade de itens, versão e persistência.

Personalização	Representação e cuidado
Moldura	ID do catálogo, decoração e cor; não aceitar estrutura privilegiada livre
Nome	Cor, negrito, itálico e sublinhado conforme schema
Banner	Modo, cores, imagem e posição validados
Aparência	Tokens de cor do perfil, não CSS arbitrário irrestrito
Bandeiras	Identificadores conhecidos e quantidade limitada
Tags	Lista e cores associadas conforme contrato
Estudos	Assuntos e detalhes declarados; não são prova de conclusão
Redes sociais	Links/campos opcionais validados; não autorizam por si integração
Título equipado	Depende de catálogo e disponibilização ao usuário

Preferências privadas de conversa ficam separadas da apresentação pública. Atualizações versionadas ajudam a detectar edição concorrente; uma tela aberta há muito tempo não deve sobrescrever silenciosamente uma versão mais nova.

Conquistas, títulos, badges e certificados

O serviço reúne métricas como posts, comentários, pessoas ajudadas, seguidores, atividades pedagógicas e condições de eventos. O catálogo declara requisito e raridade. A reconciliação compara métricas e persiste conquistas elegíveis, em vez de confiar em uma flag enviada pelo navegador.

Título é elemento exibido no perfil. Badge histórico é outra associação do schema. Credencial pedagógica verifica requisitos de aprendizagem e integridade. Não trate os três como sinônimos, nem use posse de cosmético RPG como prova acadêmica.

Relações sociais e descoberta

FollowService [../../../src/services/follow.service.ts] coordena a relação direcional. O serviço de recomendações [../../../src/services/follow-recommendation.service.ts] seleciona sugestões conforme fontes e filtros disponíveis; sugerir não equivale a seguir automaticamente.

BlockService [../../../src/services/block.service.ts] impede interações e participa de filtros de consulta. Um teste adequado confere as duas direções e o efeito no conteúdo, chat e convites. Bloqueio não deve ser apenas uma lista decorativa em configurações.

Ranking

O sistema distingue total, período e linguagem. O ranking geral tem desempate por username e ID após XP. Rank de semana/mês precisa usar movimentações do período; não deve ser calculado por uma fração inventada do total.

Reações podem ter reversão de XP; importar personagens editoriais pode criar XP ilustrativo inicial sem equivaler a atividade pedagógica real. Isso reforça a necessidade de usar a fonte correta para cada painel e credencial.

Notificações

NotificationService [../../../src/services/notification.service.ts] cria registros por destinatário e trata consulta/estado de leitura. Lista, contador de não lidas e entrega push são etapas diferentes. Uma notificação pode continuar armazenada mesmo que o push falhe.

O registro inclui tipo, recurso e contexto para navegação. Abrir o destino volta a aplicar autorização; uma notificação não é um token de acesso a candidaturas ou conversas. Quando o recurso já mudou de estado, a interface precisa lidar com esse destino sem reexecutar automaticamente a ação original.

Bate-papo

A API de mensagens [../../../src/app/api/messages/route.ts] resolve o usuário, confere destinatário e bloqueios. Na implementação observada, envio é limitado a 30 pedidos por minuto nesse fluxo. Após gravar, tenta criar a notificação; uma falha dessa tentativa é registrada separadamente.

Histórico seleciona mensagens enviadas nas duas direções entre usuário atual e destinatário. Busca 101 linhas para construir página de até 100, devolve em ordem de apresentação e informa o próximo

cursor em header. Cache-Control: private, no-store evita tratar conversa como resposta pública cacheável.

A rota também contém compatibilidade de criação/verificação da tabela de reações e fallback de consulta sem reações. Isso é um comportamento técnico existente, não uma recomendação para substituir migrations por DDL durante requisições.

Itens salvos

O salvamento liga usuário e publicação. A lista deve continuar respeitando audiência/bloqueios e existência do conteúdo. Não é um arquivo independente do post: excluir ou restringir a publicação pode afetar sua disponibilidade para quem a salvou.

client/post-bookmarks.ts coordena a atualização do cliente entre superfícies. Esse estado compartilhado evita divergências visuais, mas o banco e a API permanecem a confirmação do salvamento.

Configurações e administração

Configurações agrupam conta, aparência, sons, idioma, bloqueados e operações de conta. Alguns efeitos são locais ou sincronizados por preferências; alterações persistentes exigem a API correspondente. Alterar tema não muda permissões.

Administração usa fontes e rotas específicas, com auditoria de ações sensíveis. Alterar papel de avaliador genericamente não substitui candidatura aprovada. Gestão de recrutador não elimina propriedade da empresa. Os registros de auditoria devem permitir identificar ator, ação e recurso sem registrar senhas ou tokens.

Quiz diário

generateDailyQuiz [../../../src/services/daily-quiz.service.ts] normaliza a data para o início do dia UTC e usa unicidade por scheduled_for. Se já existe questão, retorna a existente. Seleciona entre até 100 posts públicos recentes no intervalo previsto e usa uma questão revisada relacionada ao conteúdo; a função declara explicitamente geração sem IA.

Não havendo conteúdo elegível, pode retornar null. Isso não é o mesmo que inventar uma pergunta para preencher obrigatoriamente a interface. O dia UTC dessa seleção é diferente do dia civil de São Paulo usado no baú RPG. QuizService.validateQuizAnswer confere origem, audiência e IDs reservados antes de tratar uma tentativa como quiz comum.

Denúncias

Denúncias de conteúdo são registros para consulta e ação administrativa. A pessoa que denuncia não recebe por isso poder de exclusão ou acesso ao painel. O fluxo de moderação usa domínio e auditoria próprios.

Upload de imagens

A rota genérica de upload [../../../src/app/api/upload/route.ts] exige usuário, recebe multipart, limita o arquivo a 5 MiB e aceita raster JPEG, PNG, GIF e WebP. Confere bytes de assinatura, não apenas o

MIME informado, e usa processamento/armazenamento do fluxo. Outros endpoints de mídia podem ter contratos diferentes; não aplique esse limite automaticamente a áudio/vídeo.

Comunidade editorial

A coleção de novembro contém perfis fictícios identificados como demonstração, com vozes e estilos próprios. Usa catálogo compartilhado, IDs estáveis e importação incremental. Não fabrica curtidas, visualizações ou seguidores e não envia mensagens a membros reais para simular engajamento.

O comando community:seed sem argumentos mostra o plano. --apply altera apenas identidades reservadas e conteúdo gerenciado. É diferente de prisma db seed, que limpa dados no seed de desenvolvimento. Consulte a documentação da coleção [../../DEMO_COMMUNITY.md].

Avaliação de código, duelos e revisão humana

Contratos independentes

Contexto	Entrada avaliada	Critério	Efeito possível
Lição	Resposta da etapa	Gabarito/contrato configurado	Progresso e primeiro crédito elegível
Exercício	Código de função/programa	Casos de teste e retorno esperado	Execução, submissão e primeira conclusão
Duelo competitivo	Soluções dos participantes	Correção e comparação ordenada	Resultado, revisão e XP de vitória
Mundinho	Programa de cenário	Traço e estruturas executadas	Golpe, turno e combate
Revisão humana	Parecer autorizado	Elegibilidade e contrato do parecer	Registro de decisão e recompensa prevista

Não existe uma única função de “nota de programação” que decide todos esses contextos. O mesmo código pode estar correto para um contrato e errado para outro.

Execução de código

code-execution/server.ts [../../../src/lib/code-execution/server.ts] prepara a linguagem e escolhe o caminho de execução. JS usa sandbox QuickJS; TS envolve transformação antes da execução correspondente. Outras linguagens dependem dos provedores e configurações disponíveis, como Judge0 e Wandbox.

O preparo pode envolver um programa incompleto numa estrutura esperada pela linguagem: classe Main, main, imports ou cabeçalhos. Isso melhora a usabilidade, mas não transforma qualquer fragmento em programa correto. O retorno distingue ok, output, error, unavailable e tempo de execução.

Os adaptadores usam timeout e tratam resposta do provedor. Falha HTTP, indisponibilidade e erro de compilação são situações diferentes. O servidor não deve executar código do aluno dentro do processo normal com eval irrestrito.

Harness e testes

exercises/evaluator.ts [../../../src/lib/exercises/evaluator.ts] constrói o programa de avaliação, prepara invocações e identifica resultados por marcadores. Saída extra do aluno não deve substituir o resultado estruturado esperado pelo avaliador.

Casos de teste têm entrada, saída esperada, ordem e indicação de sigilo. run usa prévia pública; submissão pode incluir ocultos. O DTO público não entrega gabarito de teste oculto. Um caso também pode especificar invocação, portanto o contrato não precisa ser apenas “ler stdin e imprimir”.

Para interpretar uma falha, siga: nome/assinatura da função → entrada → transformação → tipo do retorno → comparação → execução/parse → disponibilidade. Console correto com retorno incorreto é um exemplo comum de programa que ainda não cumpre o contrato.

Convite e compromisso competitivo

O duelo competitivo é diferente da arena privada do Mundinho. As constantes atuais definem 72 horas para convite direto, 24 horas para uma janela pública quando autorizada e duas horas para a partida após início. Manutenção trata expiração e encerramento conforme estado.

Aceitar estabelece compromisso de participação. Recusar/ignorar convite não deve ser contabilizado como abandono de uma partida aceita. A política controla sequência de abandonos e cooldown de 24 horas após três consecutivos. Envio válido no fluxo correspondente reseta a sequência conforme a política implementada.

Executar e submeter

A rota de execução oferece feedback de testes. A rota de solução registra submissão competitiva, com usuário pertencente ao duelo e estado/prazo compatível. Não basta enviar duelId e um vencedor desejado.

O registro de uma execução e a escolha da melhor submissão são responsabilidades diferentes. chooseBestDuelSubmission [../../../src/lib/duels/resolution-policy.ts] seleciona a solução representativa de cada usuário com critérios que consideram validade e correção, em vez de tomar simplesmente o último texto salvo.

Ordem da decisão automática

1. Indisponibilidade do juiz é tratada como condição operacional própria.

2. Ausência de submissões ou ausência de resultados válidos pode encerrar sem vencedor.

3. Correção e proporção de testes aprovados antecedem eficiência.

4. Se resultados equivalentes ainda não são ambos aceitos, pode haver empate técnico.

5. Se ambos foram aceitos e a sintaxe normalizada coincide, encaminha revisão humana.

6. Para soluções aceitas diferentes, complexidade estimada só compara quando há confiança prevista nas duas.

7. Linhas úteis/instruções sintáticas são o critério posterior.

8. Persistindo equivalência, encerra em empate automático.

Runtime e score histórico podem compor dados explicativos, mas não substituem essa ordem. O documento de política [../../quality/duel-judging-policy.md] aprofunda o algoritmo vigente.

Limites da análise

Normalização sintática reduz diferenças de formatação e representação, mas igualdade normalizada não prova plágio. Estimativa estática de Big-O não é uma prova formal e pode ter confiança limitada. Linhas úteis não medem sozinhas legibilidade. Tempo em provedores distintos não é comparação controlada de desempenho.

Esses limites precisam aparecer também na apresentação acadêmica: “o sistema estima e compara conforme a política” é uma descrição precisa; “o sistema prova a melhor solução possível” não é.

Persistência e recompensas

resolution.ts [../../../src/lib/duels/resolution.ts] coordena resolução e gravação. Atualização condicional do estado e transação protegem contra fechamentos concorrentes. XP de vitória é 50, concedido uma vez pelo fluxo de resultado; derrota não é débito de XP.

Notificações e entrega push não devem ser usadas como prova de que o resultado foi ou não gravado. Recuperar uma notificação pendente não reabre um duelo nem concede uma segunda recompensa.

Candidatura a avaliador

EvaluatorService.checkEligibility [../../../src/services/evaluator.service.ts] verifica progresso real da trilha ou XP suficiente. A política atual exige uma trilha completa ou pelo menos 1.000 XP. A candidatura também exige motivação normalizada de no mínimo 80 caracteres e tecnologias elegíveis.

Aprovação é ato administrativo com justificativa e registros correspondentes. Não é consequência automática de escrever “sou avaliador” na bio ou equipar um título. Rejeição, nova candidatura e moderação têm estados próprios no serviço.

Reputação e autorização para revisar

A reputação inicial no fluxo de aprovação é 100. Os limites da política são 80 para faixa ativa e 60 para suspensão. Avisos e reativação alteram o estado segundo o serviço; o papel nominal não elimina a checagem desse estado.

Antes do parecer, o serviço confere identidade, papel, situação, tecnologia e conflito. Participante não pode avaliar seu próprio duelo. Os campos numéricos e o feedback devem respeitar o contrato. Revisão concluída rende 10 XP ao avaliador no fluxo de recompensa autorizado.

Verificações recomendadas

Cenário	Resultado a conferir
Correta versus incorreta	Correção prevalece sobre código menor
Aceitas com normalização igual	Revisão humana, sem acusação automática
Diferentes com eficiência equivalente	Empate quando os critérios se esgotam

Cenário	Resultado a conferir
Compilador indisponível	Não inventar derrota pedagógica/abandono
Dois fechamentos simultâneos	Um resultado e um crédito elegível
Terceiro enviando solução	Acesso negado
Avaliador participante	Conflito rejeitado
Avaliador suspenso ou sem tecnologia	Parecer não autorizado

Veja sequências [sequencias.md] e código comentado [codigo-comentado.md] para acompanhar rotas, juiz, política e gravação.

Vagas, recrutamento, eventos e hackathons

Empresa e autorização

A empresa tem dono (owner_id). Vaga pertence à empresa; candidatura pertence ao candidato e à vaga. O papel RECRUITER habilita uma categoria de operação, mas a autorização concreta depende da empresa.

requireCompanyAccess [../../../src/lib/mobile/company-access.ts] consulta a empresa, retorna 404 quando não existe e exige administrador ou recrutador proprietário. Apesar da pasta mobile, esse helper também participa de operações web.

JobService.createCompany cria a empresa não verificada e promove uma conta comum a recrutador no fluxo observado, sem substituir automaticamente papéis privilegiados por um papel menor. Isso não significa que criar empresa dá acesso às empresas dos outros.

Listagem e detalhe da vaga

listJobs [../../../src/services/job.service.ts] monta filtros para busca, nível, modalidade, contrato, tecnologia, empresa e estado. Sem estado explícito, usa vagas abertas. Ordena por criação e inclui dados públicos selecionados da empresa, etapas e contagem de candidaturas.

getJobById agrega empresa e etapas ordenadas. Quando recebe usuário identificado, inclui a candidatura daquela pessoa. A listagem pública não deve se transformar em exportação de feedback interno e avaliações de todos os candidatos.

Criação de vaga

O serviço recebe título, descrição, nível, tecnologias, modalidade, localização, contrato, faixa salarial, requisitos, benefícios e etapas opcionais. A validação e autorização da rota precisam ocorrer antes de aceitar esses dados como operação do gestor.

Quando as etapas não são fornecidas, o serviço dispõe de etapas padrão como inscrição, desafio técnico, duelo e avaliação. A existência de uma etapa nomeada não significa que a plataforma realiza automaticamente cada interação externa: entrevista e oferta podem ser estados de acompanhamento.

Os valores permitidos ficam nos enums de schema.prisma [../../../prisma/schema.prisma], incluindo JobLevel, JobModality, JobContract, JobStatus, ApplicationStatus e RecruitmentStageType.

Candidatura

applyForJob confere vaga existente, estado aberto e ausência de candidatura duplicada. A unicidade usuário/vaga também está no banco. O registro inicial usa APPLIED e a primeira etapa quando ela existe.

A candidatura não implica contratação, aprovação em prova ou conclusão de todas as etapas. Esses estados dependem dos atos posteriores registrados.

Movimentação do funil

getJobApplications exige recrutador/administrador e requireCompanyAccess. A lista inclui perfil selecionado do candidato e etapa. Conhecer o ID da vaga não é suficiente para acessar sua lista privada.

updateApplicationStage grava mudança de etapa/status, feedback e nota técnica junto com auditoria. A rota participa da verificação de ator, empresa e etapa compatível. Não documente o método isolado como se ele conferisse toda a fronteira HTTP sozinho.

Depois do commit, o serviço tenta notificar o candidato. Falha dessa tentativa não significa que a movimentação foi revertida. Repetir a ação somente porque um push não apareceu pode produzir outra mudança indevida.

Estados não equivalem a ordem obrigatória universal

APPLIED, IN_REVIEW, TECHNICAL_CHALLENGE, DUEL, EVALUATION, INTERVIEW, OFFER e REJECTED são alternativas do modelo de candidatura. A existência do enum não prova um motor rígido que percorre automaticamente todos os valores nessa ordem. Observe o que a rota permite e o que o gestor escolhe no fluxo.

Eventos e hackathons

Event guarda tipo, datas, criador, vínculo opcional com empresa, nível mínimo, capacidade e recompensa declarada. Hackathon é um dos tipos possíveis. O status efetivo considera datas, evitando que um evento com término passado continue parecendo futuro apenas pelo estado persistido.

EventService [../../../src/services/event.service.ts] trata consulta, criação, participação, cancelamento e exclusão. Valores padrão de criação observados incluem nível mínimo 1 e recompensa declarada 250 quando não fornecida. Recompensa declarada não é crédito automático na inscrição.

Inscrição e capacidade

participate usa transação e lock por evento. Consulta pessoa, evento e participação existente. Uma participação já existente é devolvida sem duplicação. Para nova inscrição, confere encerramento, capacidade e nível calculado pelo XP oficial.

O lock protege a última vaga disputada por duas inscrições simultâneas. Uma checagem de contagem fora da transação não ofereceria a mesma proteção. EventParticipant começa com score zero.

Cancelamento rejeita evento concluído conforme o status efetivo; exclusão exige criador ou administrador no fluxo. A quantidade de participantes carregada num detalhe/classificação pode ser limitada e não deve ser confundida com a contagem total da base.

Desafios do evento

event-challenges.ts [../../../src/lib/event-challenges.ts] define o contrato: conjunto com 3–20 desafios; título de 3–120 caracteres; enunciado de 10–10.000; resposta esperada de 1–20.000; de 1–5 exemplos por desafio. Exemplos limitam entrada e saída a 5.000 caracteres e explicação a 2.000.

EventChallengeService [../../../src/services/event-challenge.service.ts] entrega lista ordenada e oculta resposta esperada de quem não pode gerenciar. Substituição do conjunto exige autorização e ocorre em transação; o fluxo rejeita alterações depois do encerramento.

Essa resposta esperada não é automaticamente um conjunto de testes públicos/ocultos do juiz de exercícios. Não foi estabelecido neste manual um ciclo completo de equipes, submissão de repositório e classificação automática de hackathon apenas pela existência dessa tabela.

Dependência de ambiente

Schema e migration versionada descrevem a estrutura prevista. Antes de operar desafios de eventos, confira migrations do ambiente; trabalhos anteriores registraram pendência específica dessa área. Uma documentação de domínio não deve afirmar que a tabela já foi aplicada em todo banco.

Dados adequados para demonstração

Prepare uma empresa do demonstrador, uma vaga aberta, etapas e um candidato separado. Mostre uma candidatura e a movimentação por gestor autorizado. Para eventos, use datas coerentes, capacidade e nível que deixem claro por que uma inscrição é aceita ou recusada.

Não movimente candidaturas de usuários reais para simular apresentação. Prefira ambiente e identidades de demonstração controlados. As sequências [sequencias.md] relacionam esses fluxos aos serviços e à persistência.

Contratos, segurança, falhas e operação

Caminho de uma requisição

Interface → rota → identidade → validação → autorização do recurso → regra de domínio → persistência/efeitos → serialização → resposta. Nem todos os arquivos seguem exatamente a mesma distribuição, mas essas responsabilidades precisam ser localizadas antes de alterar o fluxo.

apiHandler [../../../src/lib/api-handler.ts] envolve rotas compatíveis, resolve sessão quando habilitado, oferece contexto e padroniza parte dos erros. Também cria request ID e associa informações de rate limit à execução por AsyncLocalStorage. A presença do wrapper não garante que a rota exigiu usuário: confira a condição do próprio endpoint.

Identidade e banco

auth-session.ts [../../../src/lib/auth-session.ts] participa da leitura de sessão; auth.ts [../../../src/lib/auth.ts] resolve o usuário de domínio e oferece exigências de acesso. A implementação pode usar Prisma e fallback pela API de dados quando a conexão do banco falha em determinadas condições.

O ID validado pelo provedor precisa corresponder ao usuário usado no domínio. Metadata editável de perfil, nome de exibição e papel enviado no corpo não são prova de autorização. Checagens de administrador, avaliador, dono e participante pertencem ao servidor.

O runtime prioriza DATABASE_URL quando configurada; DIRECT_URL atende fluxos Prisma/administrativos e fallback previsto. Uma porta do banco inacessível e uma sessão inválida são falhas diferentes, mesmo quando ambas impedem uma página de carregar.

Contratos públicos e internos

Um modelo Prisma descreve o registro persistido. Um DTO público é uma seleção segura e útil de dados. Exemplos de dados internos que exigem cuidado: soluções, respostas esperadas ocultas, feedback privado de candidatura, tokens, preferências privadas de chat e detalhes operacionais.

TypeScript ajuda durante desenvolvimento. Zod e verificações de domínio validam a entrada real. Mesmo após schema aprovado, podem faltar autorização, saldo, capacidade ou pré-requisito. Nunca trate safeParse(...).success como aprovação completa da ação.

Erros HTTP

Status	Interpretação típica neste código	O que conferir
400	Forma/valor de entrada inválido	Schema, tipo e campos exigidos
401	Sessão ausente/inválida	Identidade e recuperação de acesso
403	Sem permissão ou requisito	Propriedade, papel, nível, pré-requisito
404	Recurso não encontrado para o fluxo	ID, existência e política de exposição
409	Conflito de estado, capacidade ou operação	Versão, duplicidade, encerramento e concorrência
429	Limite de pedidos atingido	Janela e headers de limite
500	Erro não classificado no wrapper	Logs associados ao request ID
503	Dependência indisponível	Executor, conexão ou serviço externo

O catálogo extraído de erros [erros-do-codigo.md] apresenta literais encontrados na fonte. Não é uma promessa de que toda rota sempre usa o mesmo envelope: há endpoints que retornam NextResponse.json diretamente. Consulte o adaptador concreto.

Concorrência e idempotência

Operação	Recurso disputado	Proteção relevante
Crédito pedagógico	Total/histórico do usuário	Transação, lock learning:<id> e primeira conclusão
Baú	Evento de resgate diário	Chave única usuário/dia e extrato
Capacidade de evento	Quantidade de vagas do evento	Lock por evento e checagem na transação
Fechamento de duelo	Estado e recompensa da partida	Atualização condicional e transação
Sala multiplayer	Participantes, versão e turno	Locks de sala/usuário e revalidação
Importação editorial	Conteúdo da coleção	Namespace, IDs estáveis e lock do publicador

Uma transação protege suas operações de banco, não a Internet inteira. Compilar código, fazer upload ou entregar push pode falhar independentemente. Não mantenha um lock de banco durante espera de compilador se o fluxo pode executar externamente e revalidar ao voltar.

Rede e recuperação

Erros recuperáveis precisam preservar o que já foi confirmado. Uma submissão antiga que volta depois de mudar o turno não deve causar dano novo. Uma notificação que falhou não deve desfazer a candidatura confirmada. Uma execução indisponível não deve conceder progresso nem consumir uma chance pedagógica como se o aluno tivesse errado.

O cliente deve distinguir enviar, aguardando, confirmado, falhou e estado desatualizado. “Tente de novo” só é seguro quando a operação reconhece repetição ou quando a interface consegue identificar que o primeiro pedido não foi confirmado.

Rate limit, logs e auditoria

ratelimit.ts usa Redis quando configurado e fallback local por instância. Fallback local não é um limite global distribuído entre todas as instâncias. As rotas escolhem seus limites; os exemplos de 30/min de lição e mensagens não são uma regra universal para todas as APIs.

Logs operacionais ajudam a depurar indisponibilidade. SecurityAuditEvent registra operações sensíveis do domínio. Os registros devem permitir investigação sem copiar credenciais ou todo o conteúdo privado. Request ID ajuda a correlacionar uma resposta com a execução que a produziu.

Banco, migrations e seeds

schema.prisma e migrations descrevem a evolução. A situação do schema local não comprova aplicação em produção. Confira histórico/checksum e ambiente antes de migrar. Não substitua esse processo por um reset de produção para “alinhar tudo”.

O seed de desenvolvimento atual limpa tabelas. A comunidade editorial tem importador incremental próprio. Ambos podem criar dados, mas seus escopos são radicalmente diferentes. Para documentação, não há necessidade de executar nenhum dos dois.

Testes e significado das evidências

Verificação	O que demonstra	O que não demonstra sozinha
Typecheck	Compatibilidade estática dos tipos compilados	Contratos reais de rede e autorização completa
Teste unitário	Comportamento isolado nos casos cobertos	Banco real e concorrência entre processos
Integração com rollback	Consultas/regras no banco configurado	Homologação irrestrita da produção
Build	Geração do pacote no ambiente local	Publicação e funcionamento de provedores externos
Screenshot/interação local	Apresentação e comportamento observado	Todos os dispositivos físicos
Status de deploy	Provedor concluiu a implantação	Todos os fluxos autenticados foram testados

Esta tarefa documental não precisa repetir testes caros da aplicação quando não altera sua execução. Os controles pertinentes são consistência de fontes, links, trechos, diagramas e ausência de credenciais no material produzido.

Mobile

O cliente Expo usa contratos e estado próprios em apps/mobile. Os endpoints mobile reaproveitam partes do domínio e têm adaptadores de sessão e serialização. O documento gerado do Mundinho precisa ser atualizado quando muda a experiência embarcada; mudar apenas Markdown não exige regenerar arte ou APK.

Manutenção da documentação

1. Atualize explicação e regra identificada.
2. Ajuste o catálogo de casos de uso se o fluxo mudou.
3. Atualize a sequência e os recortes associados.
4. Regenere inventários e diagramas derivados.
5. Confira links e leia uma amostra dos trechos contra a fonte.
6. Registre a data/base e o que foi efetivamente verificado.

Não copie .env.local, connection strings com senha, service keys ou tokens para exemplos. Nomes de variáveis bastam para explicar a operação.

Código comentado e rastreável

Índice [README.md]. Gerado por scripts/build-system-handbook.mjs [../../../scripts/build-system-handbook.mjs]. Base de código: 6a2bfce. Comentários explicativos são editoriais; os trechos e modelos vêm dos arquivos indicados.

Cada seção explica uma responsabilidade e mostra até 48 linhas iniciais de uma declaração real. Quando há truncamento, o comportamento restante deve ser acompanhado no arquivo completo e no capítulo. Linhas omitidas não são código substituto. Exemplos pedagógicos escritos no manual são identificados separadamente.

apiHandler

Envolve adaptadores HTTP, associa request ID, resolve contexto e classifica parte dos erros. O wrapper não exige sozinho sessão para todos os recursos.

Fonte: src/lib/api-handler.ts [../../../src/lib/api-handler.ts]. Declaração nas linhas 21–117; recorte 21–68.

```
export function apiHandler(
  handler: (req: Request, ctx: ApiContext) => Promise<Response>,
  options: { resolveSession?: boolean } = {}
) {
  return async (req: Request, segmentData: { params: Promise<any> }) => {
    const requestId = crypto.randomUUID();
    const store: RateLimitStore = {};

    return rateLimitStorage.run(store, async () => {
      try {
        let session = null;
        if (options.resolveSession !== false) {
          try {
            session = await getAuthUser();
          } catch {
            session = null;
          }
        }

        // Execute the actual route logic
        const response = await handler(req, {
          session,
          params: segmentData.params,
        });

        // Attach Rate Limit headers if set during execution
        if (store.limit !== undefined) {
          response.headers.set('X-RateLimit-Limit', String(store.limit));
        }
        if (store.remaining !== undefined) {
          response.headers.set('X-RateLimit-Remaining', String(store.remaining));
        }
        if (store.reset !== undefined) {
          response.headers.set('X-RateLimit-Reset', String(Math.ceil(store.reset / 1000)));
        }

        response.headers.set('X-Request-Id', requestId);
        return response;
```

```
    } catch (e: any) {
      let statusCode = 500;
      let responsePayload: { error: string; message: string; details?: any } = {
        error: 'INTERNAL_ERROR',
        message: 'Internal server error',
      };

      if (e instanceof ZodError) {
        statusCode = 400;
        const firstMessage =
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

requireAuth

Converte a exigência de uma identidade em contrato de servidor; a autorização do recurso ainda pertence ao fluxo específico.

Fonte: src/lib/auth.ts [../../../src/lib/auth.ts]. Declaração nas linhas 262–268; recorte 262–268.

```
export async function requireAuth(_req?: Request) {
  const user = await getAuthUser();
  if (!user) {
    throw new UnauthorizedError('UNAUTHORIZED', 'Autenticação necessária');
  }
  return user;
}
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

deriveKnowledgeStatus

Preserva estados já conquistados e distingue pré-requisito obrigatório de recomendação.

Fonte: src/lib/learning/progress.ts [../../../src/lib/learning/progress.ts]. Declaração nas linhas 15–35; recorte 15–35.

```
export function deriveKnowledgeStatus({
  persistedStatus,
  requiredPrerequisiteStatuses,
  isNextRecommended,
}: StatusInput): KnowledgeProgressStatus {
  if (
    persistedStatus === 'MASTERED' ||
    persistedStatus === 'COMPLETED' ||
    persistedStatus === 'IN_PROGRESS'
  ) {
    return persistedStatus;
  }

  const hasMissingRequirement = requiredPrerequisiteStatuses.some(
    (status) => !isKnowledgeCompleted(status)
  );

  if (hasMissingRequirement) return 'NOT_STARTED';
  if (isNextRecommended) return 'RECOMMENDED';
  return 'AVAILABLE';
}
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

calculatePathProgress

Conta nós concluídos, calcula percentual e procura a próxima indicação sem inventar conclusão.

Fonte: src/lib/learning/progress.ts [../../../src/lib/learning/progress.ts]. Declaração nas linhas 48–68; recorte 48–68.

```
export function calculatePathProgress(
  path: PathInput,
  statuses: ReadonlyMap<string, KnowledgeProgressStatus>
): LearningPathSummary {
  const completedNodes = path.nodeIds.filter((nodeId) => {
    const status = statuses.get(nodeId) ?? 'NOT_STARTED';
    return isKnowledgeCompleted(status);
  }).length;
  const totalNodes = path.nodeIds.length;
  const nextRecommendedNodeId =
    path.nodeIds.find((nodeId) => !isKnowledgeCompleted(statuses.get(nodeId) ?? 'NOT_STARTED')) ??
    null;


  return {
    ...path,
    completedNodes,
    totalNodes,
    progressPercent: totalNodes === 0 ? 0 : Math.round((completedNodes / totalNodes) * 100),
    nextRecommendedNodeId,
  };
}
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

requireLessonAccess

Confere pré-requisitos da lição e conserva revisão de conclusão anterior legítima.

Fonte: src/lib/learning/lesson-progress.ts [../../../src/lib/learning/lesson-progress.ts]. Declaração nas linhas 18–45; recorte 18–45.

```
export async function requireLessonAccess(userId: string, lesson: Lesson) {
  const learning = getLearningLesson(lesson.id);
  if (!learning) return;
  const needed = learning.prerequisites
    .map((id) => getLearningLesson(id))
    .filter((l) => l !== null);
  if (!needed.length) return;
  const rows = await prisma.quizAttempt.findMany({
    where: {
      user_id: userId,
      is_correct: true,
      quiz_id: {
        in: [...needed.flatMap((l) => l.steps.map((s) => s.id)), ...lesson.steps.map((s) => s.id)],
      },
    },
    select: { quiz_id: true },
  });
  const done = new Set(rows.map((r) => r.quiz_id));
  // Keep legitimate already-completed lessons reviewable after a curriculum update.
```

```
  if (isLessonComplete(learning, done)) return;
  const missing = needed.filter((l) => !isLessonComplete(l, done));
  if (missing.length)
    throw new AppError(
      'LESSON_LOCKED',
      `Conclua primeiro: ${missing.map((l) => l.title).join(', ')}.`,
      403
    );
}
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

assessLessonStep

Despacha a avaliação conforme o tipo da atividade; schema válido não significa resposta correta.

Fonte: src/lib/learning/assessment.ts [../../../src/lib/learning/assessment.ts]. Declaração nas linhas 19–112; recorte 19–66.

```
export async function assessLessonStep(lesson: Lesson, step: LessonStep, answer: LessonAnswer) {
  const outcome = (isCorrect: boolean, details?: string) => ({
    isCorrect,
    details,
    output: '',
    message: isCorrect ? 'Resposta validada.' : 'Revise a resposta e tente novamente.',
  });
  switch (step.type) {
    case 'concept_explanation':
      return outcome(true);
    case 'multiple_choice':
    case 'output_prediction':
      return outcome(answer.selectedOption === step.correctOptionIndex);
    case 'code_completion':
      return outcome(
        Boolean(step.blanks?.length) &&
          step.blanks!.every((b) =>
            b.expected.some((e) => e.trim() === (answer.blanks?.[b.id] ?? '').trim())
          )
      );
    case 'drag_drop':
      return outcome(
        Boolean(step.expectedBlockTokens?.length) &&
          JSON.stringify(answer.tokens) === JSON.stringify(step.expectedBlockTokens)
      );
    case 'ordering':
      return outcome(
        Boolean(step.orderItems?.length) &&
          JSON.stringify(answer.order) ===
            JSON.stringify(
              [...step.orderItems!].sort((a, b) => a.correctIndex - b.correctIndex).map((x) => x.id)
            )
      );
    case 'matching':
      return outcome(
        Boolean(step.matchingPairs?.length) &&
          step.matchingPairs!.every((p) => answer.pairs?.[p.left] === p.right)
      );
    case 'terminal':
      return outcome(
        Boolean(step.terminalExpected) && answer.command?.trim() === step.terminalExpected?.trim()
      );
```

```
    case 'code_editor':
    case 'debug':
    case 'boss_challenge': {
      if (!answer.code?.trim()) return outcome(false, 'Escreva seu código antes de executar.');
      if (step.evaluation) {
        const cases = step.evaluation.cases.filter((c) => answer.action === 'submit' || !c.hidden);
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

publicLesson

Remove gabaritos e elementos internos antes de entregar o catálogo ao aluno.

Fonte: src/lib/learning/assessment.ts [../../../src/lib/learning/assessment.ts]. Declaração nas linhas 115–135; recorte 115–135.

```
export function publicLesson(lesson: Lesson): Lesson {
  return {
    ...lesson,
    steps: lesson.steps.map((step) => {
      const {
        solutionCode: _solution,
        checkCode: _check,
        evaluation: _evaluation,
        correctOptionIndex: _answer,
        expectedBlockTokens: _tokens,
        terminalExpected: _terminal,
        ...safe
      } = step;
      return {
        ...safe,
        blanks: step.blanks?.map((b) => ({ ...b, expected: [] })),
        orderItems: step.orderItems?.map((item) => ({ ...item, correctIndex: -1 })),
      };
    }),
  };
}
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

saveLessonAssessment

Identifica primeira conclusão sob transação/lock e reúne progresso, XP e fechamento da lição.

Fonte: src/lib/learning/lesson-progress.ts [../../../src/lib/learning/lesson-progress.ts]. Declaração nas linhas 46–124; recorte 46–93.

```
export async function saveLessonAssessment(
  userId: string,
  lesson: Lesson,
  step: LessonStep,
  isCorrect: boolean,
  selectedIndex: number
) {
  return learningTransaction(async (tx) => {
    // Same per-user lock across lessons, quizzes and exercise rewards; handles different activities
concurrently.
    await tx.$executeRaw`SELECT pg_advisory_xact_lock(hashtext(${`learning:${userId}`}))`;
    const where = { user_id_quiz_id: { user_id: userId, quiz_id: step.id } };
    const attempts = await tx.quizAttempt.findMany({
```

```
      where: { user_id: userId, quiz_id: { in: lesson.steps.map((s) => s.id) } },
      select: { quiz_id: true, is_correct: true },
    });
    const previous = attempts.find((attempt) => attempt.quiz_id === step.id);
    const firstCompletion = isCorrect && !previous?.is_correct;
    const xpEarned = firstCompletion ? Math.max(0, step.xp) : 0;
    if (!previous)
      await tx.quiz.upsert({
        where: { id: step.id },
        create: {
          id: step.id,
          question: step.question ?? step.instruction ?? step.title,
          options: step.options ?? ['Avaliado no servidor'],
          correct_index: step.correctOptionIndex ?? 0,
          is_daily: false,
        },
        update: {},
      });
    if (!previous)
      await tx.quizAttempt.create({
        data: {
          user_id: userId,
          quiz_id: step.id,
          selected_index: selectedIndex,
          is_correct: isCorrect,
          xp_earned: xpEarned,
        },
      });
    else if (firstCompletion)
      await tx.quizAttempt.update({
        where,
        data: { selected_index: selectedIndex, is_correct: true, xp_earned: xpEarned },
      });
    const xpResult =
      xpEarned > 0
        ? await awardXPInTransaction(
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

evaluateExerciseCode

Coordena harness, execução e leitura de resultados dos casos de teste.

Fonte: src/lib/exercises/evaluator.ts [../../../src/lib/exercises/evaluator.ts]. Declaração nas linhas 139–207; recorte 139–186.

```
export async function evaluateExerciseCode(input: {
  code: string;
  language: string;
  functionName: string;
  testCases: EvaluatorTestCase[];
}): Promise<ExerciseEvaluation> {
  const { code, marker } = buildExerciseHarness(input.code, input.functionName, input.testCases);
  const execution = await executeCode(code, input.language);
  const parsedResults = execution.ok ? parseHarnessOutput(execution.output, marker) : null;

  if (
    !execution.ok ||
    !parsedResults ||
    parsedResults.length !== input.testCases.length ||
    new Set(parsedResults.map((result) => result?.id)).size !== input.testCases.length ||
```

```
    parsedResults.some(
      (result) =>
        !result ||
        typeof result.passed !== 'boolean' ||
        !input.testCases.some((test) => test.id === result.id)
    )
  ) {
    let hiddenCounter = 0;
    return {
      ok: false,
      passed: false,
      passedTests: 0,
      totalTests: input.testCases.length,
      tests: input.testCases.map((testCase) => ({
        id: testCase.id,
        label: testCase.is_hidden ? `Caso oculto ${++hiddenCounter}` : testCase.label,
        hidden: testCase.is_hidden,
        passed: false,
      })),
      consoleOutput: stripHarnessOutput(execution.output, marker),
      error: execution.error ?? 'A execução não produziu resultados de teste válidos.',
      executionMs: execution.executionMs,
      unavailable: execution.unavailable === true || execution.ok,
    };
  }

  const resultsById = new Map(parsedResults.map((result) => [result.id, result]));
  let hiddenCounter = 0;
  const tests = input.testCases.map((testCase) => {
    const result = resultsById.get(testCase.id);
    const label = testCase.is_hidden ? `Caso oculto ${++hiddenCounter}` : testCase.label;

    return {
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

recordExerciseSubmission

Registra a submissão e aplica a política de conclusão/recompensa, separada de uma simples execução.

Fonte: src/lib/exercises/repository.ts [../../../src/lib/exercises/repository.ts]. Declaração nas linhas 229–389; recorte 229–276.

```
export async function recordExerciseSubmission(input: {
  exercise: {
    id: string;
    knowledge_node_id: string;
    base_xp: number;
    language?: import('@prisma/client').Language;
  };
  userId: string;
  assistanceMode: AssistanceMode;
  code: string;
  evaluation: ExerciseEvaluation;
}) {
  // Provider failures are not learner attempts. Keep this guard in the repository
  // so every caller is protected before a transaction can mutate progression.
  requireValidEvaluation(input.evaluation);
  const lockKey = `learning:${input.userId}`;
```

```
  return learningTransaction(async (transaction) => {
    await transaction.$executeRaw`SELECT pg_advisory_xact_lock(hashtext(${lockKey}))`;

    const previousCompletion = await transaction.exerciseSubmission.findFirst({
      where: {
        user_id: input.userId,
        exercise_id: input.exercise.id,
        first_completion: true,
      },
      select: { id: true },
    });
    const firstCompletion =
      input.evaluation.ok &&
      input.evaluation.totalTests > 0 &&
      input.evaluation.passed &&
      !previousCompletion;
    const xpEarned = firstCompletion
      ? calculateExerciseXp(input.exercise.base_xp, input.assistanceMode)
      : 0;
    const publicTests = input.evaluation.tests.filter((test) => !test.hidden);
    const hiddenTests = input.evaluation.tests.filter((test) => test.hidden);

    const submission = await transaction.exerciseSubmission.create({
      data: {
        exercise_id: input.exercise.id,
        user_id: input.userId,
        assistance_mode: input.assistanceMode,
        status: getSubmissionStatus(input.evaluation),
        code: input.code,
        passed_public: publicTests.filter((test) => test.passed).length,
        total_public: publicTests.length,
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

recordAssistanceEvent

Registra o uso de ajuda no contrato do exercício.

Fonte: src/lib/exercises/repository.ts [../../../src/lib/exercises/repository.ts]. Declaração nas linhas 391–405; recorte 391–405.

```
export async function recordAssistanceEvent(input: {
  exerciseId: string;
  userId: string;
  assistanceMode: AssistanceMode;
  eventType: 'HINT_OPENED' | 'DOCUMENTATION_OPENED';
}) {
  await prisma.learningEvent.create({
    data: {
      user_id: input.userId,
      exercise_id: input.exerciseId,
      event_type: input.eventType,
      assistance_mode: input.assistanceMode,
    },
  });
}
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

calculateExerciseXp

Explicita os multiplicadores atuais por modo; eles não aumentam XP na configuração atual.

Fonte: src/lib/exercises/progression.ts [../../../src/lib/exercises/progression.ts]. Declaração nas linhas 17–19; recorte 17–19.

```
export function calculateExerciseXp(baseXp: number, mode: AssistanceMode) {
  return Math.round(Math.max(0, baseXp) * XP_MULTIPLIER[mode]);
}
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

deriveCompletedNodeStatus

Conclui o nó sem declarar domínio independente que não foi medido.

Fonte: src/lib/exercises/progression.ts [../../../src/lib/exercises/progression.ts]. Declaração nas linhas 33–42; recorte 33–42.

```
export function deriveCompletedNodeStatus(input: {
  completedExercises: number;
  totalExercises: number;
  mastery: number;
}): KnowledgeProgressStatus {
  if (input.totalExercises <= 0 || input.completedExercises < input.totalExercises)
    return 'IN_PROGRESS';
  // Completion is not independently measured mastery. Assistance is self-reported.
  return 'COMPLETED';
}
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

prepareCode

Adapta fragmentos de determinadas linguagens ao programa exigido pelo provedor, sem assegurar correção pedagógica.

Fonte: src/lib/code-execution/server.ts [../../../src/lib/code-execution/server.ts]. Declaração nas linhas 35–68; recorte 35–68.

```
export function prepareCode(
  code: string,
  language: string,
  provider: 'judge0' | 'wandbox'
): string {
  const trimmed = code.trim();

  switch (language) {
    case 'java': {
      if (/class\s+\w+/i.test(trimmed)) {
        return provider === 'wandbox' ? trimmed.replace(/public\s+class\s+/gi, 'class ') : trimmed;
      }
      const classDeclaration = provider === 'wandbox' ? 'class Main' : 'public class Main';
      return `${classDeclaration} {\n  public static void main(String[] args) {\n${indent(trimmed)}\n  }\n}`;
    }
    case 'go':
      return /package\s+main/i.test(trimmed)
        ? trimmed
```

```
      : `package main\n\nimport "fmt"\n\nfunc main() {\n${indent(trimmed)}\n}`;
    case 'cpp':
    case 'c':
      return /#include|main\s*\(/i.test(trimmed)
        ? trimmed
        : `#include <iostream>\nusing namespace std;\n\nint main() {\n${indent(trimmed)}\n  return 0;\n}`;
    case 'kotlin':
      return /fun\s+main\s*\(/i.test(trimmed) ? trimmed : `fun main() {\n${indent(trimmed)}\n}`;
    case 'swift':
      return /import\s+Foundation/i.test(trimmed) ? trimmed : `import Foundation\n\n${trimmed}`;
    case 'rust':
      return /fn\s+main\s*\(/i.test(trimmed) ? trimmed : `fn main() {\n${indent(trimmed)}\n}`;
    default:
      return trimmed;
  }
}
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

executeCode

Escolhe o caminho de execução e distingue resultado de indisponibilidade.

Fonte: src/lib/code-execution/server.ts [../../../src/lib/code-execution/server.ts]. Declaração nas linhas 214–265; recorte 214–261.

```
export async function executeCode(code: string, language: string): Promise<ServerExecutionResult> {
  const aliases: Record<string, string> = { js: 'javascript', ts: 'typescript', py: 'python' };
  const normalized = aliases[language.toLowerCase()] ?? language.toLowerCase();
  const judge0LanguageId = JUDGE0_LANGUAGES[normalized];
  const wandboxLanguage = WANDBOX_LANGUAGES[normalized];

  if (!judge0LanguageId && !wandboxLanguage) {
    return {
      ok: false,
      output: '',
      error: `Linguagem "${language}" não suportada.`,
      executionMs: 0,
    };
  }

  if (normalized === 'javascript' || normalized === 'typescript') {
    const executableCode =
      normalized === 'typescript'
        ? ts.transpileModule(code, {
            compilerOptions: {
              target: ts.ScriptTarget.ES2020,
              module: ts.ModuleKind.None,
              strict: false,
            },
            reportDiagnostics: false,
          }).outputText
        : code;
    const localResult = await runJavaScriptInSandbox(executableCode);
    if (localResult) return localResult;
  }

  if (judge0LanguageId) {
    const result = await runOnJudge0(prepareCode(code, normalized, 'judge0'), judge0LanguageId);
    if (result) return result;
```

```
  }

  if (wandboxLanguage && normalized !== 'kotlin' && normalized !== 'swift') {
    const result = await runOnWandbox(
      prepareCode(code, normalized, 'wandbox'),
      wandboxLanguage.compiler
    );
    if (result) return result;
  }

  return {
    ok: false,
    output: '',
    error: 'Serviço de execução indisponível no momento. Tente novamente em instantes.',
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

awardXPInTransaction

Centraliza o crédito confirmado no contexto da transação recebida.

Fonte: src/lib/xp.ts [../../../src/lib/xp.ts]. Declaração nas linhas 19–113; recorte 19–66.

```
export async function awardXPInTransaction(
  tx: Prisma.TransactionClient,
  userId: string,
  language: Language | null | undefined,
  amount: number
) {
  if (!Number.isSafeInteger(amount) || amount < 0) throw new Error('INVALID_XP_AMOUNT');
  await tx.$executeRaw`SELECT pg_advisory_xact_lock(hashtext(${`learning:${userId}`}))`;
  // Buscar usuário primeiro para calcular a ofensiva geral
  const user = await tx.user.findUnique({
    where: { id: userId },
    select: { streak_days: true, last_active_at: true },
  });

  const now = new Date();
  const newStreakDays = calculateNextStreak(user?.streak_days ?? 0, user?.last_active_at, now);

  // 1. Atualizar o total_xp, streak_days, last_active_at do usuário
  const updatedUser = await tx.user.update({
    where: { id: userId },
    data: {
      total_xp: {
        increment: amount,
      },
      streak_days: newStreakDays,
      last_active_at: now,
    },
  });

  await recordPeriodXp(tx, userId, language, amount);

  if (!language)
    return {
      xpEarned: amount,
      language: null,
      newXp: updatedUser.total_xp,
```

```
      newLevel: calculateLevel(updatedUser.total_xp).level,
      newStreak: newStreakDays,
      totalXp: updatedUser.total_xp,
    };

  // 2. Buscar ou criar a trilha da linguagem
  const trail = await tx.languageTrail.findUnique({
    where: {
      user_id_language: { user_id: userId, language },
    },
  });
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

calculateLevel

Usa limites iniciais e incrementos crescentes; nível não é sinônimo de faixa visual.

Fonte: src/lib/learning/rewards.ts [../../../src/lib/learning/rewards.ts]. Declaração nas linhas 13–29; recorte 13–29.

```
export function calculateLevel(xp: number) {
  const thresholds = [0, 500, 800, 1100, 1500, 2000];
  const value = Number.isFinite(xp) ? Math.max(0, xp) : 0;
  for (let i = 1; i < thresholds.length; i++) {
    if (value < thresholds[i])
      return { level: i, prevLevelXp: thresholds[i - 1], nextLevelXp: thresholds[i] };
  }
  let level = 6,
    prevLevelXp = 2000,
    increment = 600;
  while (value >= prevLevelXp + increment) {
    prevLevelXp += increment;
    level++;
    increment += 100;
  }
  return { level, prevLevelXp, nextLevelXp: prevLevelXp + increment };
}
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

reconcileCredentialsInTransaction

Confronta evidências e metas e persiste snapshots elegíveis de credencial.

Fonte: src/lib/credentials/repository.ts [../../../src/lib/credentials/repository.ts]. Declaração nas linhas 49–221; recorte 49–96.

```
export async function reconcileCredentialsInTransaction(
  tx: Prisma.TransactionClient,
  userId: string
): Promise<CredentialCollection> {
  const [user, correctAttempts, submissions, exercises, paths, earned] = await Promise.all([
    tx.user.findUnique({ where: { id: userId }, select: { username: true, avatar_config: true } }),
    tx.quizAttempt.findMany({
      where: { user_id: userId, is_correct: true },
      select: { quiz_id: true },
    }),
```

```
    tx.exerciseSubmission.findMany({
      where: { user_id: userId, status: 'PASSED' },
      select: {
        exercise_id: true,
        total_public: true,
        passed_public: true,
        total_hidden: true,
        passed_hidden: true,
      },
    }),
    tx.exercise.findMany({
      where: { is_published: true },
      select: {
        id: true,
        title: true,
        language: true,
        knowledge_node_id: true,
        estimated_minutes: true,
        _count: { select: { test_cases: true } },
      },
    }),
    tx.learningPath.findMany({
      where: { is_published: true },
      select: {
        id: true,
        slug: true,
        title: true,
        description: true,
        nodes: {
          orderBy: { position: 'asc' },
          select: {
            knowledge_node: {
              select: { id: true, title: true, language: true, is_published: true },
            },
          },
        },
      },
    }),
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

getEarnedCredentials

Lê credenciais públicas válidas sem conceder novas por causa de visita ao perfil.

Fonte: src/lib/credentials/repository.ts [../../../src/lib/credentials/repository.ts]. Declaração nas linhas 232–241; recorte 232–241.

```
export async function getEarnedCredentials(userId: string): Promise<CredentialRecord[]> {
  const rows = await prisma.learningCredential.findMany({
    where: { userId, revokedAt: null },
    select: recordSelect,
    orderBy: { issuedAt: 'desc' },
  });
  return rows
    .map(publicRecord)
    .filter((record): record is CredentialRecord => record?.status === 'valid');
}
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

getPublicCredential

Limita formato do identificador e retorna o registro de verificação correspondente.

Fonte: src/lib/credentials/repository.ts [../../../src/lib/credentials/repository.ts]. Declaração nas linhas 243–250; recorte 243–250.

```
export async function getPublicCredential(id: string): Promise<CredentialRecord | null> {
  if (!/^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$/i.test(id)) return null;
  const row = await prisma.learningCredential.findUnique({ where: { id }, select: recordSelect });
  if (!row) return null;
  const record = publicRecord(row);
  if (!record) return null;
  return record;
}
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

create

Cria publicação a partir de contrato e anexos autorizados, com identificação de pedidos repetidos.

Fonte: src/services/post.service.ts [../../../src/services/post.service.ts]. Declaração nas linhas 22–185; recorte 22–69.

```
  async create(userId: string, data: CreatePostInput, clientId?: string) {
    const { title, body, language, code, image_url, type, audience, attachments, music, poll } =
      data;

    // Idempotency check: if client_id already exists for this user, return existing post
    if (clientId) {
      const existing = await prisma.post.findFirst({
        where: { author_id: userId, client_id: clientId },
      });
      if (existing) {
        return { post: existing, xpResult: null };
      }
    }

    let leveledUp = false;
    const result = await prisma.$transaction(async (tx) => {
      const mediaIds = attachments.map((item) => item.url.split('/').at(-1)!);
      if (new Set(mediaIds).size !== mediaIds.length)
        throw new ValidationError(
          'INVALID_MEDIA',
          'Um mesmo anexo não pode ser adicionado duas vezes.'
        );
      if (mediaIds.length) {
        const media = await tx.postMedia.findMany({
          where: { id: { in: mediaIds }, owner_id: userId, post_id: null },
        });
        if (
          media.length !== mediaIds.length ||
          attachments.some((item) => {
            const file = media.find((file) => file.id === item.url.split('/').at(-1));
            return (
              !file ||
              (item.type === 'audio'
                ? !file.content_type.startsWith('audio/')
                : item.type === 'gif'
```

```
              ? file.content_type !== 'image/gif'
              : !file.content_type.startsWith('image/'))
          );
        })
      )
        throw new ValidationError(
          'INVALID_MEDIA',
          'Um anexo está indisponível. Envie o arquivo novamente.'
        );
    }
    // 1. Create Post
    const post = await tx.post.create({
      data: {
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

getFeed

Monta consulta por filtros, cursor e visibilidade antes de apresentar a página.

Fonte: src/services/post.service.ts [../../../src/services/post.service.ts]. Declaração nas linhas 187–337; recorte 187–234.

```
async getFeed(
  userId: string | null,
  params: {
    language?: string;
    languages?: string[];
    search?: string;
    author?: string;
    filter?: string;
    sort?: 'recent' | 'popular';
    cursor?: string;
    limit?: number;
    likedBy?: string;
    answeredBy?: string;
  }
) {
  const {
    language,
    languages,
    search,
    author,
    filter,
    sort,
    cursor,
    limit = 10,
    likedBy,
    answeredBy,
  } = params;

  const whereClause: any = {};

  if (filter === 'following') {
    if (!userId) {
      whereClause.id = { equals: '__anonymous_following_feed__' };
    } else {
      whereClause.author = {
        followers: {
          some: { followerId: userId },
```

```
        },
      };
    }
  }

  if (language) {
    whereClause.language = language as Language;
  }
  if (author) {
    whereClause.author = { username: author };
  }
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

visiblePostWhere

Compõe restrições de autor e audiência para consulta; conhecer ID não concede acesso.

Fonte: src/services/post-access.service.ts [../../../src/services/post-access.service.ts]. Declaração nas linhas 7–20; recorte 7–20.

```
export function visiblePostWhere(viewerId: string | null): Prisma.PostWhereInput {
  return {
    author: visibleUserWhere(viewerId),
    OR: [
      { audience: 'everyone' },
      ...(viewerId
        ? [
            { author_id: viewerId },
            { audience: 'followers', author: { followers: { some: { followerId: viewerId } } } },
          ]
        : []),
    ],
  };
}
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

toggleReaction

Serializa transições de reação e seus efeitos/reversões de XP.

Fonte: src/services/reaction.service.ts [../../../src/services/reaction.service.ts]. Declaração nas linhas 13–136; recorte 13–60.

```
  async toggleReaction(userId: string, postId: string, type: string | null) {
    if (type !== null && !VALID_REACTIONS.includes(type)) {
      throw new Error('INVALID_REACTION_TYPE');
    }

    return await prisma.$transaction(async (tx) => {
      // Serialize reaction transitions with all XP writes for this user.
      await tx.$executeRaw`SELECT pg_advisory_xact_lock(hashtext(${`learning:${userId}`}))`;
      // 1. Verify post exists
      const post = await tx.post.findUnique({
        where: { id: postId },
      });

      if (!post) {
```

```
      throw new NotFoundError('POST_NOT_FOUND', 'Postagem não encontrada');
    }

    await BlockService.assertCanInteract(userId, post.author_id, tx);
    await assertPostAudience(post, userId, tx);

    // 2. Fetch existing reaction
    const existingReaction = await tx.reaction.findUnique({
      where: {
        post_id_user_id: {
          post_id: postId,
          user_id: userId,
        },
      },
    });

    if (type === null) {
      if (existingReaction) {
        // Remove reaction
        await tx.reaction.delete({
          where: {
            post_id_user_id: {
              post_id: postId,
              user_id: userId,
            },
          },
        });

        // Reverse the reaction reward in this transaction. The general award
        // helper intentionally rejects negative amounts.
        const user = await tx.user.findUniqueOrThrow({ where: { id: userId } });
        await tx.user.update({
          where: { id: userId },
          data: { total_xp: { decrement: Math.min(2, Math.max(0, user.total_xp)) } },
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

toggleFollow

Representa a relação direcional de seguir, distinta de amizade mútua.

Fonte: src/services/follow.service.ts [../../../src/services/follow.service.ts]. Declaração nas linhas 8–76; recorte 8–55.

```
async toggleFollow(followerId: string, followingId: string, ensureFollowing = false) {
  if (followerId === followingId) {
    throw new ValidationError('SELF_FOLLOW', 'Você não pode seguir a si mesmo');
  }

  const result = await prisma.$transaction(async (tx) => {
    await BlockService.assertCanInteract(followerId, followingId, tx);
    if (ensureFollowing) {
      await tx.$queryRaw`SELECT id FROM "User" WHERE id = ${followerId} FOR UPDATE`;
    }
    // 1. Verify target user exists
    const targetUser = await tx.user.findUnique({
      where: { id: followingId },
    });
```

```
    if (!targetUser) {
      throw new NotFoundError('USER_NOT_FOUND', 'Usuário não encontrado');
    }

    // 2. Check if already following
    const existingFollow = await tx.follow.findUnique({
      where: {
        followerId_followingId: {
          followerId,
          followingId,
        },
      },
    });

    if (existingFollow) {
      // A stale recommendation or a retry must never undo an existing follow.
      if (ensureFollowing) return { following: true };
      // Unfollow
      await tx.follow.delete({
        where: {
          followerId_followingId: {
            followerId,
            followingId,
          },
        },
      });
      logger.info('User unfollowed', { followerId, followingId });
      return { following: false };
    } else {
      // Follow
      await tx.follow.create({
        data: {
          followerId,
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

getUserNotifications

Consulta notificações do destinatário aplicando a política do serviço.

Fonte: src/services/notification.service.ts [../../../src/services/notification.service.ts]. Declaração nas linhas 138–322; recorte 138–185.

```
  async getUserNotifications(userId: string, cursor?: string, limit = 10, useCursor = true) {
    const where = {
      userId,
      OR: [{ actorId: null }, { actor: { is: visibleUserWhere(userId) } }],
      ...buildCursorWhere(cursor, 'createdAt'),
    };

    const takeVal = useCursor ? limit + 1 : undefined;

    const notifications = await prisma.notification.findMany({
      where,
      orderBy: [{ createdAt: 'desc' }, { id: 'desc' }],
      take: takeVal,
      include: {
        actor: {
          select: {
            id: true,
```

```
          username: true,
          avatar_url: true,
          avatar_config: true,
          total_xp: true,
        },
      },
    },
  });

  const jobIds = [
    ...new Set(
      notifications
        .filter((notification) => notification.resourceType === 'JOB' && notification.resourceId)
        .map((notification) => notification.resourceId!)
    ),
  ];
  // One query for the page, bounded to this candidate's applications.
  const jobs = jobIds.length
    ? await prisma.job.findMany({
        where: { id: { in: jobIds } },
        select: {
          id: true,
          title: true,
          company: { select: { name: true } },
          applications: {
            where: { user_id: userId },
            take: 1,
            select: { status: true, stage: { select: { title: true } } },
          },
        },
      })
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

markAllAsRead

Marca somente os registros do destinatário informado pelo fluxo autorizado.

Fonte: src/services/notification.service.ts [../../../src/services/notification.service.ts]. Declaração nas linhas 324–334; recorte 324–334.

```
async markAllAsRead(userId: string) {
  await prisma.notification.updateMany({
    where: {
      userId,
      read: false,
    },
    data: {
      read: true,
    },
  });
},
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

saveProfileCustomization

Valida preferências, versão e itens permitidos para gravar a identidade do titular.

Fonte: src/services/profile-customization.service.ts [../../../src/services/profile-customization.service.ts]. Declaração nas linhas 469–660; recorte 469–516.

```
export async function saveProfileCustomization(
  userId: string,
  input: unknown
): Promise<ProfileCustomizationState> {
  const parsed = saveProfileCustomizationSchema.parse(input);
  const existing = await prisma.profileCustomization.findUnique({ where: { userId } });
  const current = parseProfilePreferences(existing?.preferences);
  parsed.preferences = normalizeProfilePreferenceAliases(parsed.preferences, current);
  const bannerImage = parsed.preferences.banner.imageUrl;
  if (
    bannerImage &&
    bannerImage !== current.banner.imageUrl &&
    !isOwnedProfileUpload(bannerImage, userId)
  )
    throw new ValidationError(
      'BANNER_IMAGE_NOT_OWNED',
      'Envie uma imagem pela sua conta para usar no banner.'
    );
  // Client metadata is never authoritative, even when the track ID stays the same.
  if (parsed.preferences.music) {
    if (current.music?.trackId === parsed.preferences.music.trackId) {
      parsed.preferences.music = current.music;
    } else {
      const resolved = await resolveSpotifyTrack(parsed.preferences.music.trackId);
      // Availability diagnostics are transport-only and must not contaminate
      // the strict persisted preferences schema.
      parsed.preferences.music = {
        trackId: resolved.trackId,
        title: resolved.title,
        artist: resolved.artist,
        artworkUrl: resolved.artworkUrl,
        ...(resolved.album !== undefined ? { album: resolved.album } : {}),
        ...(resolved.durationMs !== undefined ? { durationMs: resolved.durationMs } : {}),
      };
    }
  }
  const catalog = await getProfileCatalog();
  await prisma.$transaction(
    async (tx) => {
      const { achievements } = await reconcileInTransaction(tx, userId, catalog);
      const row = await tx.profileCustomization.findUnique({ where: { userId } });
      if ((row?.version || 0) !== parsed.version)
        throw new AppError(
          'PROFILE_VERSION_CONFLICT',
          'Seu perfil mudou em outra sessão. Recarregue antes de salvar.',
          409,
          { currentVersion: row?.version || 0 }
        );
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

dayKey

Define dia civil em São Paulo para recibos e sequências; não é relógio livre do navegador.

Fonte: src/lib/mundinho/rules.ts [../../../src/lib/mundinho/rules.ts]. Declaração nas linhas 12–19; recorte 12–19.

```
export function dayKey(now: Date) {
  return new Intl.DateTimeFormat('en-CA', {
    timeZone: 'America/Sao_Paulo',
    year: 'numeric',
    month: '2-digit',
    day: '2-digit',
  }).format(now);
}
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

resolveTurn

Resolve combate de criatura com efeitos do golpe e contra-ataque; PvP usa alternância própria.

Fonte: src/lib/mundinho/rules.ts [../../../src/lib/mundinho/rules.ts]. Declaração nas linhas 45–66; recorte 45–66.

```
export function resolveTurn(
  hp: number,
  enemyHp: number,
  correct: boolean,
  skill: string,
  level: number,
  equipment: { maxHp: number; protection: number } = { maxHp: 100, protection: 0 }
) {
  const move = SKILLS.find((s) => s.id === skill) ?? SKILLS[0];
  const nextEnemy = Math.max(0, enemyHp - (correct ? move.damage : 0));
  const nextHp = Math.max(
    0,
    Math.min(equipment.maxHp, hp + (correct ? move.heal : 0)) -
      (nextEnemy > 0 ? Math.max(1, (correct ? 9 : 18) + level * 2 - equipment.protection) : 0)
  );
  return {
    hp: nextHp,
    enemyHp: nextEnemy,
    status:
      nextEnemy === 0 ? ('won' as const) : nextHp === 0 ? ('lost' as const) : ('active' as const),
  };
}
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

equipmentStats

Deriva bônus de itens efetivamente possuídos e equipados em slots válidos.

Fonte: src/lib/mundinho/equipment.ts [../../../src/lib/mundinho/equipment.ts]. Declaração nas linhas 142–163; recorte 142–163.

```
export function equipmentStats(
  inventory: readonly string[],
  equipped: Record<string, string>
): CombatStats {
  let luck = 0,
    protection = 0,
    vitality = 0;
  for (const item of EQUIPMENT) {
    if (!inventory.includes(item.id) || equipped[item.slot] !== item.id) continue;
    luck += item.luck;
```

```
    protection += item.protection;
    vitality += item.vitality;
  }
  luck = Math.min(25, luck);
  return {
    maxHp: 100 + Math.min(50, vitality),
    protection: Math.min(8, protection),
    luck,
    directChance: hasBugHunterTitle(inventory, equipped) ? 100 : Math.min(100, 75 + luck),
    dropChance: 50 + luck,
  };
}
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

credit

Liga crédito ao recibo único no extrato para reconhecer repetição.

Fonte: src/lib/mundinho/repository.ts [../../../src/lib/mundinho/repository.ts]. Declaração nas linhas 155–160; recorte 155–160.

```
async function credit(tx: Tx, userId: string, key: string, label: string, amount: number) {
  if (await tx.rpgTransaction.findUnique({ where: { userId_key: { userId, key } } })) return false;
  await tx.rpgTransaction.create({ data: { userId, key, label, amount } });
  await tx.rpgPlayer.update({ where: { userId }, data: { coins: { increment: amount } } });
  return true;
}
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

adventure

Coordena ações autenticadas do RPG; a função completa contém os ramos de economia e batalha.

Fonte: src/lib/mundinho/repository.ts [../../../src/lib/mundinho/repository.ts]. Declaração nas linhas 308–702; recorte 308–355.

```
export async function adventure(userId: string, action?: Action): Promise<AdventureState> {
  let codeRun: CodeRun | undefined;
  let notice: string | undefined;
  // Execute outside the transaction: an external compiler must not hold the player row lock.
  if (action?.type === 'battle-code') {
    const battle = await prisma.rpgBattle.findFirst({ where: { id: action.battle, userId } });
    if (
      !battle ||
      battle.status !== 'active' ||
      battle.turn !== action.turn ||
      battle.questionId !== action.question
    )
      return {
        ...(await adventure(userId)),
        notice: 'A batalha foi atualizada. Este turno já não está ativo.',
      };
    const challenge = codeChallenge(battle.language as BattleLanguage, battle.questionId!);
    if (!challenge || challenge.id !== battle.questionId)
      fail('Este turno não é um desafio de código.');
    const prepared = instrumentChallengeSource(
      battle.language as BattleLanguage,
      action.code,
```

```
      challenge
    );
    if (prepared.error) {
      codeRun = { passed: false, trace: [], error: prepared.error };
    } else {
      const execution = await executeCode(
        challengeProgram(battle.language as BattleLanguage, prepared.source, challenge),
        challengeExecutionLanguage(battle.language as BattleLanguage)
      );
      if (!execution.ok) {
        codeRun = {
          passed: false,
          trace: [],
          retryable: true,
          error: `${execution.error ?? 'Não foi possível executar o código.'}${execution.output &&
execution.output !== '(sem output)' ? `\n${execution.output.slice(0, 1800)}` : ''}\nSeu turno não foi consumido.
Revise ou tente novamente.`,
        };
      } else codeRun = evaluateTrace(execution.output, challenge);
    }
  }
  return prisma.$transaction(
    async (tx) => {
      // The database clock is authoritative; clients never supply dates, prices, damage or rewards.
      const [clock] = await tx.$queryRaw<{ now: Date }[]>`SELECT CURRENT_TIMESTAMP AS now`;
      const now = clock.now;
      let chestAwarded: boolean | undefined;
      await tx.rpgPlayer.upsert({
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

validArenaPosition

Confere limites de movimento e deslocamento compatível com tempo decorrido.

Fonte: src/lib/mundinho/arena-rules.ts [../../../src/lib/mundinho/arena-rules.ts]. Declaração nas linhas 32–39; recorte 32–39.

```
export function validArenaPosition(previous: ArenaPosition, next: ArenaPosition, elapsed: number) {
  // Network jitter gets a small allowance; a client cannot jump across the room.
  const distance = Math.hypot(next.x - previous.x, next.y - previous.y);
  return (
    arenaCanWalk(next.x, next.y) &&
    distance <= Math.min(600, (Math.max(0, elapsed) / 1000) * 180 + 48)
  );
}
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

chooseArenaAttack

Trava sorteio/seleção do exercício e resolve golpe direto com o snapshot de equipamento.

Fonte: src/lib/mundinho/arena-rules.ts [../../../src/lib/mundinho/arena-rules.ts]. Declaração nas linhas 89–112; recorte 89–112.

```
export function chooseArenaAttack(
  match: ArenaMatch,
  roll: number,
  now: Date,
```

```
  skill: ArenaSkill = 'strike'
): ArenaMatch {
  if (arenaNeedsExercise(match)) return match;
  match = { ...match, skill };
  const stats = match.stats![match.actorId];
  if (!directAttack(stats, roll))
    return {
      ...match,
      exerciseTurn: match.turn,
      log: [...match.log, 'Resolva o exercício para confirmar este golpe.'].slice(-8),
    };
  const damage = arenaAttackDamage(match);
  const move = SKILLS.find((s) => s.id === skill)!;
  return advanceArenaTurn(
    match,
    true,
    now,
    `${move.name}: ${stats.directChance === 100 ? 'Caça-bugs / sorte máxima!' : 'Golpe direto!'} ${damage} de
dano no adversário.${move.heal > 0 ? ' Recupera até 20 de vida.' : move.heal < 0 ? ' Recebe 8 de desgaste.' :
''}`
  );
}
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

advanceArenaTurn

Aplica dano, cura e desgaste, verifica encerramento e alterna o ator do turno.

Fonte: src/lib/mundinho/arena-rules.ts [../../../src/lib/mundinho/arena-rules.ts]. Declaração nas linhas 43–78; recorte 43–78.

```
export function advanceArenaTurn(
  match: ArenaMatch,
  correct: boolean,
  now: Date,
  message: string
): ArenaMatch {
  const opponent = arenaOpponent(match, match.actorId);
  const move = SKILLS.find((s) => s.id === match.skill) ?? SKILLS[0];
  const damage = arenaAttackDamage(match);
  const hp = {
    ...match.hp,
    [opponent]: Math.max(0, match.hp[opponent] - (correct ? damage : 0)),
    [match.actorId]: Math.max(
      0,
      Math.min(
        match.stats?.[match.actorId]?.maxHp ?? 100,
        match.hp[match.actorId] + (correct ? move.heal : 0)
      )
    ),
  };
  const finished = hp[opponent] === 0 || hp[match.actorId] === 0 || match.turn >= 19;
  const ids = Object.keys(hp);
  const winnerId = hp[ids[0]] === hp[ids[1]] ? null : hp[ids[0]] > hp[ids[1]] ? ids[0] : ids[1];
  return {
    ...match,
    hp,
    status: finished ? 'finished' : 'active',
    winnerId: finished ? winnerId : null,
    actorId: opponent,
```

```
    turn: match.turn + 1,
    exerciseTurn: -1,
    skill: undefined,
    deadline: new Date(now.getTime() + ARENA_TURN_MS).toISOString(),
    log: [...match.log, message].slice(-8),
  };
}
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

arena

Integra identidade, participação, presença, turnos, compilação e persistência da sala.

Fonte: src/lib/mundinho/arena.ts [../../../src/lib/mundinho/arena.ts]. Declaração nas linhas 240–664; recorte 240–287.

```
export async function arena(userId: string, action: ArenaAction): Promise<ArenaState> {
  let codeRun: CodeRun | undefined;
  const admissionIds = [userId];
  if (action.type === 'accept') {
    const invitation = await prisma.rpgArenaRoom.findUnique({
      where: { id: action.roomId },
      select: { hostId: true },
    });
    if (invitation) admissionIds.push(invitation.hostId);
  }
  if (action.type === 'run') {
    await rateLimit(`arena-run:${userId}`, { limit: 12, window: '1 m', endpoint: '/api/mundinho' });
    // Compilers run without holding a room lock. The turn is checked again below.
    const room = await prisma.rpgArenaRoom.findUnique({ where: { id: action.roomId } });
    if (!room) throw new NotFoundError();
    member(room, userId);
    const match = matchOf(room);
    if (
      !match ||
      match.id !== action.matchId ||
      match.status !== 'active' ||
      match.actorId !== userId ||
      match.turn !== action.turn
    )
      invalid('Este turno já mudou. Atualize a arena.');
    if (!arenaNeedsExercise(match)) invalid('Escolha atacar antes de executar um exercício.');
    const language = (
      room.hostId === userId ? room.hostLanguage : room.guestLanguage
    ) as BattleLanguage;
    const challenges = codeChallenges(language, match.level);
    const question = challenges[(match.scenario + Math.floor(match.turn / 2)) % challenges.length];
    if (question.id !== action.questionId) invalid('O desafio deste turno mudou.');
    const prepared = instrumentChallengeSource(language, action.code, question);
    if (prepared.error) codeRun = { passed: false, trace: [], error: prepared.error };
    else {
      const result = await executeCode(
        challengeProgram(language, prepared.source, question),
        challengeExecutionLanguage(language)
      );
      codeRun = result.ok
        ? evaluateTrace(result.output, question)
        : {
            passed: false,
            trace: [],
```

```
            retryable: true,
            error: result.error ?? 'Executor indisponível. Seu turno foi preservado.',
          };
    }
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

chooseBestDuelSubmission

Escolhe a submissão representativa de um participante segundo validade e critérios da política.

Fonte: src/lib/duels/resolution-policy.ts [../../../src/lib/duels/resolution-policy.ts]. Declaração nas linhas 82–108; recorte 82–108.

```
export function chooseBestDuelSubmission(
  submissions: ComparableSubmission[],
  userId: string,
  acceptedOnly = false,
  language = 'TS'
) {
  return (
    submissions
      .filter((s) => s.user_id === userId && (!acceptedOnly || s.status === 'ACCEPTED'))
      .sort((a, b) => {
        if (valid(a) !== valid(b)) return valid(a) ? -1 : 1;
        if (valid(a) && valid(b)) {
          const gap = correctness(a, b);
          if (gap) return -gap;
          if (a.status === 'ACCEPTED' && b.status === 'ACCEPTED') {
            const efficiency = compareEfficiency(metrics(a, language), metrics(b, language));
            if (efficiency) return -efficiency;
          }
        }
        const outage =
          Number(b.status === 'JUDGE_UNAVAILABLE') - Number(a.status === 'JUDGE_UNAVAILABLE');
        return (
          outage || a.created_at.getTime() - b.created_at.getTime() || a.id.localeCompare(b.id)
        );
      })[0] ?? null
  );
}
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

decideDuelResolution

Aplica a ordem de correção, igualdade normalizada e desempate, distinguindo revisão de empate.

Fonte: src/lib/duels/resolution-policy.ts [../../../src/lib/duels/resolution-policy.ts]. Declaração nas linhas 109–178; recorte 109–156.

```
export function decideDuelResolution(input: {
  first: ComparableSubmission | null;
  second: ComparableSubmission | null;
  reason: 'automatic_score' | 'deadline_score';
  language?: string;
}): DuelResolutionDecision {
  const { first, second } = input,
    language = input.language ?? 'TS';
  const comparison: Comparison = {
```

```
    criterion: 'none',
    first: first ? metrics(first, language) : null,
    second: second ? metrics(second, language) : null,
  };
  const base = {
    scoreGap: Math.abs((first?.score ?? 0) - (second?.score ?? 0)),
    runtimeGap:
      first?.runtime_ms != null && second?.runtime_ms != null
        ? Math.abs(first.runtime_ms - second.runtime_ms)
        : null,
    comparison,
  };
  const closed = (
    reason: DuelResolutionDecision['reason'],
    winnerId: string | null = null
  ): DuelResolutionDecision => ({ ...base, kind: 'closed', reason, winnerId });
  if (first?.status === 'JUDGE_UNAVAILABLE' || second?.status === 'JUDGE_UNAVAILABLE')
    return closed('judge_unavailable');
  if (!first && !second) return closed('no_submissions');
  const a = valid(first) ? first : null,
    b = valid(second) ? second : null;
  if (!a && !b) return closed('no_valid_submissions');
  if (!a || !b) {
    comparison.criterion = 'correctness';
    return closed(input.reason, (a ?? b)!.user_id);
  }
  const correctGap = correctness(a, b);
  if (correctGap) {
    comparison.criterion = 'correctness';
    return closed(input.reason, correctGap > 0 ? a.user_id : b.user_id);
  }
  if (a.status !== 'ACCEPTED' || b.status !== 'ACCEPTED') {
    comparison.criterion = 'tie';
    return closed('technical_draw');
  }
  const canonicalA = analyzeDuelSource(a.code, language).canonical,
    canonicalB = analyzeDuelSource(b.code, language).canonical;
  if (canonicalA && canonicalA === canonicalB) {
    comparison.criterion = 'identical_code';
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

awardDuelWinnerXP

Mantém crédito de vitória associado ao resultado e sua proteção contra repetição.

Fonte: src/lib/duels/resolution.ts [../../../src/lib/duels/resolution.ts]. Declaração nas linhas 233–242; recorte 233–242.

```
export async function awardDuelWinnerXP(duelId: string, winnerId: string, language: Language) {
  return prisma.$transaction(async (tx) => {
    const claim = await tx.duel.updateMany({
      where: { id: duelId, status: 'CLOSED', winner_id: winnerId, xp_awarded_at: null },
      data: { xp_awarded_at: new Date() },
    });
    if (claim.count !== 1) return null;
    return awardXPInTransaction(tx, winnerId, language, 50);
  });
}
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

checkEligibility

Confere XP ou conclusão real de trilha para a candidatura a avaliador.

Fonte: src/services/evaluator.service.ts [../../../src/services/evaluator.service.ts]. Declaração nas linhas 30–91; recorte 30–77.

```
async checkEligibility(userId: string) {
  const user = await prisma.user.findUnique({
    where: { id: userId },
    include: {
      quiz_attempts: {
        where: { is_correct: true },
        select: { quiz_id: true },
      },
      evaluator_profile: true,
      evaluator_applications: {
        orderBy: { created_at: 'desc' },
        take: 1,
      },
    },
  });

  if (!user) {
    return { eligible: false, reason: 'Usuário não encontrado', completedTrails: [] };
  }

  const completedQuizIds = new Set(user.quiz_attempts.map((a) => a.quiz_id));
  const completedTrails: string[] = [];

  // Check each language trail
  for (const [lang, levels] of Object.entries(TRAILS_DATA)) {
    const allQuestionsCompleted = levels.every((lvl) =>
      lvl.questions.every((q) => completedQuizIds.has(q.id))
    );
    if (allQuestionsCompleted && levels.length > 0) {
      completedTrails.push(lang);
    }
  }
  // Current learning paths also award eligibility; legacy assessment IDs remain valid.
  for (const course of LEARNING_COURSES) {
    const steps = course.lessons.flatMap((lesson) =>
      lesson.steps.filter((step) => step.type !== 'concept_explanation')
    );
    if (steps.length && steps.every((step) => completedQuizIds.has(step.id))) {
      if (!completedTrails.includes(course.language)) completedTrails.push(course.language);
    }
  }

  const hasTrail = completedTrails.length >= 1;
  const hasSufficientXp = user.total_xp >= EVALUATOR_MIN_XP;
  const eligible = hasTrail || hasSufficientXp;

  const latestApplication = user.evaluator_applications[0] || null;
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

submitDuelEvaluation

Confronta autorização, situação e conflito antes do parecer humano e seus efeitos.

Fonte: src/services/evaluator.service.ts [../../../src/services/evaluator.service.ts]. Declaração nas linhas 412–569; recorte 412–459.

```
async submitDuelEvaluation(params: {
  duelId: string;
  evaluatorId: string;
  scorePlayer1: number;
  scorePlayer2: number;
  winnerId: string;
  humanFeedback: string;
  strengths: string[];
  improvements: string[];
}) {
  const {
    duelId,
    evaluatorId,
    scorePlayer1,
    scorePlayer2,
    winnerId,
    humanFeedback,
    strengths,
    improvements,
  } = params;

  const duel = await prisma.duel.findUnique({
    where: { id: duelId },
    include: { challenger: true, opponent: true },
  });

  if (!duel) throw new NotFoundError('DUEL_NOT_FOUND', 'Duelo não encontrado.');
  if (duel.status !== 'REVIEW_PENDING') {
    throw new ValidationError(
      'DUEL_NOT_PENDING',
      'Este duelo não está aguardando desempate humano.'
    );
  }
  if (duel.challenger_id === evaluatorId || duel.opponent_id === evaluatorId) {
    throw new ForbiddenError(
      'DUEL_SELF_REVIEW',
      'Participantes não podem avaliar o próprio duelo.'
    );
  }
  if (winnerId !== duel.challenger_id && winnerId !== duel.opponent_id) {
    throw new ValidationError(
      'DUEL_INVALID_WINNER',
      'O vencedor deve ser um dos participantes do duelo.'
    );
  }
  if (
    [scorePlayer1, scorePlayer2].some(
      (score) => !Number.isInteger(score) || score < 0 || score > 1000
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

requireCompanyAccess

Exige empresa existente e administrador ou recrutador proprietário.

Fonte: src/lib/mobile/company-access.ts [../../../src/lib/mobile/company-access.ts]. Declaração nas linhas 3–9; recorte 3–9.

```
export async function requireCompanyAccess(companyId: string, user: { id: string; role: string }) {
  const company = await prisma.company.findUnique({ where: { id: companyId } });
  if (!company) throw new AppError('COMPANY_NOT_FOUND', 'Empresa não encontrada.', 404);
  if (user.role !== 'ADMIN' && (user.role !== 'RECRUITER' || company.owner_id !== user.id))
    throw new AppError('FORBIDDEN', 'Você não gerencia esta empresa.', 403);
  return company;
}
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

applyForJob

Rejeita vaga inadequada/duplicata e cria candidatura inicial.

Fonte: src/services/job.service.ts [../../../src/services/job.service.ts]. Declaração nas linhas 256–299; recorte 256–299.

```
  async applyForJob(userId: string, jobId: string) {
    const existing = await prisma.jobApplication.findUnique({
      where: {
        job_id_user_id: {
          job_id: jobId,
          user_id: userId,
        },
      },
    });

    if (existing) {
      throw new Error('Você já se candidatou a esta vaga.');
    }

    const job = await prisma.job.findUnique({
      where: { id: jobId },
      include: {
        stages: { orderBy: { order: 'asc' } },
        company: true,
      },
    });

    if (!job || job.status !== 'OPEN') {
      throw new Error('Esta vaga não está mais aceitando inscrições.');
    }

    const firstStage = job.stages[0];

    const application = await prisma.jobApplication.create({
      data: {
        job_id: jobId,
        user_id: userId,
        stage_id: firstStage?.id,
        status: 'APPLIED',
      },
```

```
    include: {
      stage: true,
      job: { include: { company: true } },
    },
  });

  logger.info('User applied for job', { userId, jobId, applicationId: application.id });
  return application;
},
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

updateApplicationStage

Grava movimento do funil com auditoria e trata notificação posterior separadamente.

Fonte: src/services/job.service.ts [../../../src/services/job.service.ts]. Declaração nas linhas 304–358; recorte 304–351.

```
async updateApplicationStage(params: {
  applicationId: string;
  stageId?: string;
  status: ApplicationStatus;
  feedback?: string;
  technicalScore?: number;
  actorId?: string;
}) {
  const { applicationId, stageId, status, feedback, technicalScore } = params;

  const application = await prisma.$transaction(async (tx) => {
    const updated = await tx.jobApplication.update({
      where: { id: applicationId },
      data: {
        stage_id: stageId,
        status,
        feedback,
        technical_score: technicalScore,
      },
      include: {
        stage: true,
        user: {
          select: {
            id: true,
            username: true,
            avatar_url: true,
            total_xp: true,
            bio: true,
            github_username: true,
          },
        },
      },
    });
    await recordSecurityAudit(tx, {
      actorId: params.actorId,
      action: 'JOB_APPLICATION_UPDATED',
      resourceId: applicationId,
      metadata: { job_id: updated.job_id, status, stage_id: updated.stage_id },
    });
    return updated;
  });
```

```
  try {
    await NotificationService.create({
      userId: application.user_id,
      type: 'JOB_APPLICATION_UPDATE',
      resourceId: application.job_id,
      resourceType: 'JOB',
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

participate

Serializa a disputa de capacidade e confere encerramento, nível e participação existente.

Fonte: src/services/event.service.ts [../../../src/services/event.service.ts]. Declaração nas linhas 178–230; recorte 178–225.

```
async participate(userId: string, eventId: string) {
  return prisma.$transaction(async (tx) => {
    // Capacity is shared by every participant, so serialize joins for the event.
    await tx.$executeRaw`SELECT pg_advisory_xact_lock(hashtext(${`event-participation:${eventId}`}))`;
    const user = await tx.user.findUnique({
      where: { id: userId },
      select: { id: true, total_xp: true },
    });
    const event = await tx.event.findUnique({
      where: { id: eventId },
      include: { _count: { select: { participants: true } } },
    });

    if (!user) throw new AppError('USER_NOT_FOUND', 'Usuário não encontrado.', 404);
    if (!event) throw new AppError('EVENT_NOT_FOUND', 'Evento não encontrado.', 404);
    const existing = await tx.eventParticipant.findUnique({
      where: { event_id_user_id: { event_id: eventId, user_id: userId } },
    });
    if (existing) return existing;
    if (event.status === 'COMPLETED' || event.end_date <= new Date())
      throw new AppError('EVENT_CLOSED', 'Este evento já foi encerrado.', 409);
    if (event.max_participants && event._count.participants >= event.max_participants) {
      throw new AppError('EVENT_FULL', 'Limite máximo de participantes atingido.', 409);
    }

    const userLevel = calculateLevel(user.total_xp).level;
    if (event.min_level > 1 && userLevel < event.min_level) {
      throw new AppError(
        'EVENT_LEVEL_REQUIRED',
        `Nível mínimo necessário (${event.min_level}) não atingido. Seu nível atual é ${userLevel}.`,
        403
      );
    }

    const participant = await tx.eventParticipant.upsert({
      where: {
        event_id_user_id: {
          event_id: eventId,
          user_id: userId,
        },
      },
      update: {},
      create: {
```

```
        event_id: eventId,
        user_id: userId,
        score: 0,
      },
    });
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

replaceSet

Valida a permissão e substitui o conjunto editável de desafios em transação.

Fonte: src/services/event-challenge.service.ts [../../../src/services/event-challenge.service.ts]. Declaração nas linhas 62–105; recorte 62–105.

```
async replaceSet(
  eventId: string,
  challenges: EventChallengeInput[],
  viewer: EventChallengeViewer
) {
  const event = await findEvent(eventId);

  if (viewer.id !== event.creator_id && viewer.isAdmin !== true) {
    throw new ForbiddenError(
      'EVENT_CHALLENGE_FORBIDDEN',
      'Somente o criador do evento pode gerenciar os desafios.'
    );
  }

  if (effectiveEventStatus(event) === 'COMPLETED') {
    throw new ValidationError(
      'EVENT_COMPLETED',
      'Os desafios não podem ser alterados depois que o evento termina.'
    );
  }

  await prisma.$transaction(async (transaction) => {
    await transaction.eventChallenge.deleteMany({ where: { event_id: eventId } });
    await transaction.eventChallenge.createMany({
      data: challenges.map((challenge, index) => ({
        event_id: eventId,
        position: index + 1,
        title: challenge.title,
        statement: challenge.statement,
        language: challenge.language,
        expected_answer: challenge.expected_answer,
        examples: challenge.examples,
      })),
    });
  });

  logger.info('Event challenge set replaced', {
    eventId,
    userId: viewer.id,
    challengeCount: challenges.length,
  });

  return EventChallengeService.list(eventId, viewer);
},
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

planCommunity

Recusa colisões com contas comuns e prepara uma importação de escopo reservado.

Fonte: prisma/demo-community/publish.ts [../../../prisma/demo-community/publish.ts]. Declaração nas linhas 66–98; recorte 66–98.

```
export async function planCommunity(prisma: PrismaClient) {
  const users = await prisma.user.findMany({
    where: { username: { in: COMMUNITY_PROFILES.map((p) => p.username) } },
    select: { id: true, username: true, email: true },
  });
  for (const profile of COMMUNITY_PROFILES) {
    const asset = assets[profile.username];
    if (!asset) throw new Error(`Missing avatar and reserved emails: ${profile.username}`);
    const existing = users.find((u) => u.username === profile.username);
    if (existing && !asset.emails.includes(existing.email))
      throw new Error(`Username belongs to a non-demo account: ${profile.username}`);
    communityPreferences(profile.username);
    if (
      !DEFAULT_PROFILE_CATALOG.frames.some((f) => f.id === profile.frameId) ||
      !DEFAULT_PROFILE_CATALOG.achievements.some((a) => a.id === profile.titleId)
    )
      throw new Error(`Unknown frame or title for ${profile.username}`);
  }
  const ids = users.map((u) => u.id);
  const managedPosts = await prisma.post.count({
    where: { author_id: { in: ids }, client_id: { startsWith: `${COMMUNITY_NAMESPACE}:` } },
  });
  return {
    profiles: COMMUNITY_PROFILES.length,
    newProfiles: COMMUNITY_PROFILES.length - users.length,
    threads: COMMUNITY_THREADS.length,
    notes: COMMUNITY_NOTES.length,
    continuations: COMMUNITY_THREADS.reduce((sum, t) => sum + t.parts.length - 1, 0),
    discussionMessages: COMMUNITY_THREADS.length * 2,
    existingManagedPosts: managedPosts,
    users,
  };
}
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

publishCommunity

Atualiza perfis e publicações com namespace/IDs estáveis sem usar o seed destrutivo.

Fonte: prisma/demo-community/publish.ts [../../../prisma/demo-community/publish.ts]. Declaração nas linhas 101–309; recorte 101–148.

```
export async function publishCommunity(
  prisma: PrismaClient,
  userIds: Record<string, string>,
  initializeProgressFor: readonly string[] = []
) {
  await planCommunity(prisma);
  return prisma.$transaction(
```

```
async (tx) => {
  // Serialize publishers. Auth provisioning happens before this short database transaction.
  await tx.$executeRaw`SELECT pg_advisory_xact_lock(571139, 20261103)`;
  for (const profile of COMMUNITY_PROFILES) {
    const id = userIds[profile.username];
    if (!id) throw new Error(`Missing Auth identity for ${profile.username}`);
    const preferences = communityPreferences(profile.username);
    const frame = DEFAULT_PROFILE_CATALOG.frames.find((f) => f.id === profile.frameId)!;
    const title = DEFAULT_PROFILE_CATALOG.achievements.find((a) => a.id === profile.titleId)!;
    const existing = await tx.user.findUnique({
      where: { id },
      select: { avatar_config: true, email: true, username: true },
    });
    if (
      existing &&
      (existing.username !== profile.username ||
        !assets[profile.username].emails.includes(existing.email))
    )
      throw new Error(`Identity collision for ${profile.username}`);
    const config =
      existing?.avatar_config &&
      typeof existing.avatar_config === 'object' &&
      !Array.isArray(existing.avatar_config)
        ? existing.avatar_config
        : {};
    const visual = {
      displayName: profile.displayName,
      editorialDemo: true,
      profileStyle: {
        frame,
        title: {
          id: title.id,
          name: title.name,
          color: title.color,
          rarity: title.rarity,
          icon: title.icon,
        },
        nameStyle: preferences.nameStyle,
        flag: 'BR',
        flags: ['BR'],
      },
```

Como ler: identifique argumentos e sua origem; acompanhe validações e ramos; localize a gravação/retorno e os erros. Consulte o fluxo completo nas sequências [sequencias.md].

Referência de UML e critérios de modelagem

Referência principal: GUEDES, Gilleanes T. A. UML 2: uma abordagem prática. 3. ed. São Paulo: Novatec, 2018. ISBN 978-85-7522-644-5. Consultado no PDF fornecido pelo grupo. Os números de página abaixo são a posição no PDF de 657 páginas; podem diferir da paginação impressa.

O livro orienta a notação e a organização da documentação. As regras da Stacklyst são verificadas no código do projeto; os exemplos bancários e demais sistemas do livro não são requisitos desta plataforma. Os textos e diagramas desta revisão são próprios do projeto.

Casos de uso e sua especificação

Conforme a seção 3.4 (PDF, pp. 65-67), um caso de uso deve explicar o objetivo do ator e a resposta do sistema em linguagem compreensível. O catálogo identifica nome e ID, ator principal, pré-

condições, fluxo principal, resultado esperado e alternativas. Referências de código e regras de negócio complementam a especificação e permitem verificar a implementação. A UML admite formatos diferentes para essa documentação; a estrutura adotada é uma adaptação ao projeto.

Cada passo do catálogo deve ser lido como interação entre ator e sistema. No diagrama, as elipses representam objetivos ou comportamentos reutilizados. Tabelas, componentes React, rotas e consultas são detalhados nos modelos estruturais e nas sequências, sem transformá-los em atores humanos.

Herança dos atores

A generalização/especialização segue a seção 3.7 (PDF, pp. 70-73). Avaliador, Recrutador e Administrador especializam Usuário. A linha contínua termina em triângulo vazio voltado para Usuário, o ator geral. Um papel especializado participa das interações comuns de Usuário e acrescenta as interações autorizadas ao seu papel.

Essa generalização representa papéis de interação. Não afirma que existam subclasses TypeScript desses nomes nem concede permissões em execução: o servidor continua validando sessão, papel, propriedade e escopo. Uma mesma pessoa pode exercer mais de um papel. Cadastro, recuperação e verificação pública são interações de Usuário cujas pré-condições permitem ausência de sessão.

Inclusão, extensão e pontos de extensão

Na inclusão, o comportamento chamado é obrigatório no contexto do caso que o inclui (seção 3.8; PDF, pp. 74-75). A dependência tracejada com «include» aponta do caso base para o incluído. Na visão geral, Resolver exercícios inclui Avaliar resposta; Submeter solução de duelo inclui Executar código. A validação das entradas e a autorização antecedem a execução efetiva, conforme os fluxos alternativos documentados.

Na extensão, o comportamento adicional ocorre sob uma condição, em um ponto identificável do caso base (seções 3.9 a 3.11; PDF, pp. 76-80). A dependência tracejada com «extend» aponta da extensão para o caso base. Executar código estende Avaliar resposta quando a atividade é de código, no ponto avaliar código. Consultar ajuda estende Resolver exercícios se o usuário solicitar e o modo permitir, no ponto consultar ajuda. Atividades objetivas podem ser avaliadas sem execução de código; a ajuda não é uma etapa obrigatória de todos os exercícios.

Relações de inclusão e extensão descrevem dependência comportamental. Não indicam ordem cronológica de navegação. Autenticação é pré-condição dos fluxos protegidos; não se repete um «include Fazer login» para toda operação.

Classes e persistência

Os capítulos 2 e 4 fundamentam classes, atributos, operações e multiplicidades. A seção 4.15 (PDF, pp. 213-218) distingue classes de tabelas e discute o mapeamento para persistência. As vistas CL desta revisão são conceituais, com nomes em português e responsabilidades do domínio. A visão geral tem oito classes; as vistas específicas têm quatro a seis. Uma classe conceitual não afirma que exista uma declaração TypeScript class com o mesmo nome.

As vistas DER são recortes relacionais em notação de classes UML: dez tabelas na visão geral e quatro ou cinco nas vistas específicas. CP significa chave primária e CE, chave estrangeira. Os nomes visíveis são traduções; a tabela de correspondência identifica as tabelas e campos reais.

Campos e relações auxiliares permanecem no dicionário completo do banco. O Astah UML instalado não oferece o editor ER da edição Professional; a notação utilizada está explícita. A redução é de apresentação, sem alteração do schema.

Sequências e ligação com o código

O capítulo 7 orienta participantes, linhas de vida, mensagens, retornos e condições. As seções 7.3.3 e 7.8 (PDF, pp. 272-273 e 286-293) apoiam a leitura de respostas e alternativas. A ordem vertical mostra a progressão da interação; a posição horizontal separa responsabilidades.

Nas vistas nativas desta revisão, chamadas são mensagens UML editáveis e condições são guardas nas mensagens e notas de resposta. As respostas podem ser expressas nos valores de retorno das chamadas e em notas de resultado, dispensando setas de retorno redundantes. As sequências completas devem ser lidas junto ao fluxo textual e suas alternativas. Os recortes SQ01-A a SQ01-C e SQ11-A a SQ11-F relacionam uma parte da interação ao arquivo e às linhas reais da implementação.

O cenário principal, os caminhos alternativos e as falhas previstas precisam concordar com a especificação. Um retorno de erro não comprova sucesso, uma prévia não concede recompensa de submissão e uma repetição não deve ser documentada como novo crédito de XP. Essas são regras da implementação conferidas separadamente da referência bibliográfica.

Uso no seminário

A sequência de exposição segue a orientação da professora: protótipo, RF/RNF/RN, modelos estruturais, sequência completa, recortes com código e demonstração prática. A referência de Guedes fundamenta os modelos; a ordem, o tempo e a divisão dos integrantes provêm da orientação do seminário. Os diagramas de atividades previamente aprovados permanecem preservados.